

Homework I: Cache Reverse Engineering Across Architectures and CPU Generations

Fall 2026

Due: September 10, 2026 at 11:59 PM on Moodle

1 Objective

The goal of this homework is to reverse engineer the CPU cache hierarchy using controlled software microbenchmarks and timing measurements, then verify the inferred results using hardware performance counters where such counters are available. You will compare cache behavior across multiple Intel, AMD, and Arm microarchitectures and across CPU generations.

The central question is not only *what* each cache looks like, but also *how cache design changes across time and architectures*. After sorting the tested microarchitectures by introduction year, you will investigate whether any cache-related quantity exhibits a trend that can be modeled or extrapolated, in the spirit of Moore's observation of a systematic technology trend.

2 Learning Outcomes

By the end of this assignment, you should be able to:

- Design timing-only microbenchmarks for cache reverse engineering.
- Infer cache hierarchy, capacity, line size, associativity, inclusion/exclusion behavior, and hit/miss latency from performance transitions.
- Separate experimental inference from information obtained through architectural documentation or PMU counters.
- Use hardware performance counters to validate a microarchitectural hypothesis.
- Compare semantically similar performance events across Intel, AMD, and Arm systems.
- Construct a software-only estimator for a cache metric when PMU access is unavailable.
- Analyze cache evolution across CPU generations and formulate a defensible trend or prediction.

3 Requirements and Deliverables

- **Teamwork:** This homework is completed in **teams of exactly two students**. Both members are expected to contribute substantially to the code, experimental runs, plotting/writing, and analysis. The mandatory contribution appendix at the end of the report must document the division of work in detail.
- **Submission on Moodle — one ZIP only:** Submit by **September 10, 2026 at 11:59 PM Eastern Time**. Submit **one team submission containing one .zip archive**. The ZIP must contain the final main reverse-engineering source code for every architecture used, analysis/plotting scripts, the main raw and processed data used to support the reported results (and all required raw data described later), plots, the PDF report, and the final slide deck in PDF format. Do not upload the code, data, and report as unrelated separate submissions.
- Include complete reproduction instructions for both the ECE machines and the NC State HPC cluster.
- **GitHub sharing and first-page report link:** Maintain a GitHub repository for all code and analysis scripts, preserve the complete commit history, and grant repository access to **both the TA and the instructor**. Put the **GitHub repository URL on the first page of the report** and also in the submitted README.md. Both students must commit using GitHub accounts that can be identified in the repository history.
- **Overleaf sharing and first-page report link:** Use a shared Overleaf project for the written report and grant project access to **both the TA and the instructor**. Put the **Overleaf project URL on the first page of the report** and also in the submitted README.md.
- **All main reverse-engineering code must appear in the report body:** The body of the PDF report must contain the **complete main source-code listings actually used for the reverse-engineering experiments**, not only pseudocode, isolated fragments, GitHub links, or appendix references. Include the main code for cache-level/capacity discovery, line-size inference, associativity/conflict testing, hit and miss/next-level latency, inclusion/exclusion behavior, and the software-only cache metric, including the architecture-specific versions used for **Intel/AMD x86-64 and Arm/AArch64**. If one parameterized portable program implements several experiments, include that complete main program once and clearly identify which functions/options implement each inference. Architecture-specific timing, serialization, affinity, or measurement paths must be shown. The listings in the paper must correspond to the final code submitted in the Moodle ZIP and the frozen GitHub commit used for the results.

- **GitHub contribution evidence:** In the mandatory contribution appendix, include a screenshot of the GitHub repository's contributor/contribution plot (for example, the repository Insights/Contributors view or an equivalent GitHub contribution view) showing **both students' GitHub accounts**. The report must state each student's full name next to the corresponding GitHub username/account so the grader can identify the two contributors. The screenshot supplements, but does not replace, the detailed contribution tables and commit references.
- **Required paper title:** Write the report as a research paper titled *"Predicting Cache Evolution: The Growing Capacity and Latency Cost of CPU Caching."* The paper should use the lab-machine measurements to establish historical cache trends and make frozen predictions that are subsequently tested on the held-out Hazel HPC machines.
- Prepare a companion slide deck that summarizes the problem, methodology, key reverse-engineering logic, main results, cross-machine comparison, and conclusions.
- For every reverse-engineering task, include at least one diagram or flow chart that explains the experimental logic behind the inference and how the measurement leads to the conclusion.
- Include the mandatory team-contribution appendix described later in this homework. In addition, identify the primary contributor below every figure and table.
- Vector plots are preferred.
- **Plot line-style rule:** Remove background grid lines from result plots. For chronological/prediction plots, draw the **examined/collected measured years with solid lines and solid markers**; use a **dashed line only for future-year extrapolation/prediction beyond the last measured training-year point**. Later held-out Hazel measurements should appear with distinct markers and must not be used to redraw or refit the frozen dashed prediction.
- **AI/LLM assistance is allowed, but must be disclosed and understood.** In the report appendix, list the AI tool(s) used and the **type of help received** (for example: brainstorming an experiment, explaining a concept, debugging, generating or modifying code, plotting, writing/editing, or analysis). You remain responsible for everything you submit. You must understand all AI-assisted work and be able to explain, without relying on the AI, **how the submitted code works, how each experiment works, how the data were collected, how each plot was generated, and how each conclusion follows from the data**. Submitting AI-generated code, text, plots, or analysis that the team cannot explain is not acceptable.

4 Access, Login, and Running Instructions

Complete the access checks below **early**. Do not wait until the final week. If a login fails, send the TA your Unity ID, the hostname, the date/time, and the exact error message or a screenshot. Do not create repeated IT tickets unless the TA asks you to do so.

Getting Help and Asking Questions

Do not stay blocked on an access or experimental problem. Use the following support channels:

- **TA – Darsh Asher:** dkasher@ncsu.edu. Reach out to Darsh for homework questions, experimental-methodology questions, access coordination, and help interpreting unexpected results.
- **TA office hour: Tuesday, 2:00–3:00 PM.** Use the office/Zoom location announced on Moodle/Discord. Bring your code, plots, raw output, error messages, or a minimal example of the problem so the issue can be diagnosed efficiently.
- **Course Discord:** Join <https://discord.gg/gy8GzKPEa>. Use Discord for general homework questions, clarifications, and issues that may help other teams. Do not post passwords, private credentials, restricted machine information, or complete solution code. If the invite expires, contact the TA.
- **ECE IT – lab servers and machines:** ecehelp@ncsu.edu. Contact ECE IT for account/login problems, unavailable ECE lab machines, server hardware/OS issues, or course permissions that the TA cannot resolve. Include the hostname, your Unity ID, the time of the failure, and the exact error message.
- **NC State HPC support – Hazel:** help@ncsu.edu. Contact HPC support for Hazel authorization, scheduler/Slurm problems, HPC account issues, unavailable resources, or cluster-specific problems. Include your Unity ID, course/project information, Slurm job ID when applicable, requested CPU constraint, and the relevant error/output.

Recommended order for help: for homework logic, code, measurements, or interpretation, ask the TA/Discord first. For ECE lab infrastructure problems, use ECE IT. For Hazel/HPC infrastructure problems, use HPC support.

A. ECE/Ajorpaz Lab Machines

Course access has been requested for the enrolled students. If you enrolled late please email IT and cc the instructor to make sure your access is granted.

Host	CPU model from course slides	ISA/Architecture	Microarchitecture	Year
Sunbird	2× Xeon E5-2680 v3	To research	To research	To research
Thunderbird	Ampere Q80-30	To research	To research	To research
Skylark	2× AMD EPYC 7532	To research	To research	To research
Artemisia	2× Xeon Gold 5420+	To research	To research	To research
Charnwood	Core i7-6700	To research	To research	To research
Crux	Core i7-9700	To research	To research	To research
Ookay	Core i7-7700	To research	To research	To research
Upgrade	Core i7-8700	To research	To research	To research

Table 1: ECE/Ajorpaz lab machines used for the timing-only training/observation phase. Students must research the ISA, microarchitecture, and introduction year without consulting cache specifications before the Phase-I freeze.

1. **Connect to the NC State VPN.** Install Cisco Secure Client 5.x if needed, connect to `vpn.ncsu.edu`, and use the course/workshop VPN group specified in class (the course slide lists **8-Workshop-Temp**). Authenticate with your NC State credentials and Duo when prompted.

2. **SSH using your Unity ID.** The general form is:

```
ssh <unityid>@<hostname>.ece.ncsu.edu
```

For example:

```
ssh <unityid>@artemisia.ece.ncsu.edu
```

```
ssh <unityid>@sunbird.ece.ncsu.edu
```

Use the hostnames listed in Table 1. If one host uses a different DNS name, follow the TA/course announcement.

3. **Verify access to every assigned lab machine.** A successful login is enough for the initial check. Do not start by reading cache specifications; Phase I is timing-only.
4. **Do not reboot or reconfigure shared machines.** Use only the privileges granted to your course account. Do not disable system services, change security settings, alter global kernel parameters, or reboot a machine unless the instructor/TA explicitly authorizes it.
5. **PMU permissions are Phase II only.** If a performance-counter command is denied after Phase I has been frozen, record the exact error and contact the TA. Do not work around a permission restriction by changing system-wide settings yourself.

Useful Phase-I identification commands. You may identify the machine and topology without intentionally reading cache sizes. Examples include:

```
hostname
uname -a
grep -m1 -E 'model name|Hardware|Processor' /proc/cpuinfo
lscpu -e=CPU,CORE,SOCKET,NODE
```

Machine research requirement. Complete the research fields in Table 1. For each host you experiment with in this homework, find credible sources for:

- (a) instruction-set architecture (e.g., x86-64 or AArch64),
- (b) CPU vendor and processor family,
- (c) microarchitecture/codename,
- (d) approximate year that microarchitecture or processor generation was introduced,
- (e) fabrication/process node if reliably documented.

Use vendor documentation when possible. Record your sources. This research is for taxonomy and historical ordering; cache specifications must remain hidden from your Phase-I analysis until your timing-only values are finalized.

Year convention for every chronological plot: use the first public introduction/launch year of the **specific processor generation or microarchitecture represented by the CPU**, not the year the physical machine was purchased or installed. Choose one convention (processor-generation launch year or microarchitecture introduction year), state it once, and use it consistently for every lab and Hazel point.

B. Quiet-Hour Reservation on Lab Machines

The Ajorpaz lab machines are shared. Most experiments may not need a reservation for a core to get a clean result but just in case if interference prevents a low noise measurement, we require you to use the following sheet to run experiments on any machines (this does not apply HPC machines which are managed by the university) and check before you start your experiments if you want to make sure no one else is using that core or you are not interfering with someone else experiments. Please reserve a specific core or CPU package/socket for a one-hour interval using:

Homework I Lab Machine Core Reservation Sheet

A reservation applies only to the ECE/lab machines and does not imply exclusive ownership of the entire server. Record the reserved host, package/socket, core, and time interval in the report.

C. NC State Hazel HPC

Course project ece592f26 has been created with access for all students. More information about HPC cluster course projects is here: [HPC guide](#).

Scheduler note for Fall 2026: NC State is transitioning Hazel from LSF to Slurm. Some examples for Slurm environment is given below but you should use the live Slurm status/tools available. If your authorized project exposes only LSF resources, or a requested CPU generation is not available through Slurm, contact the TA/HPC support rather than translating the assignment to LSF on your own. Never move a benchmark to a login node as a workaround.

1. **Authorization.** Before connecting, you must be authorized for the course HPC project and comply with the NC State HPC Acceptable Use Policy. If your Unity ID is not authorized, email the hpc help desk and copy the instructor for approval.
2. **Connect to Hazel.** From a terminal:

```
ssh <unityid>@login.hpc.ncsu.edu
```


You may also use NC State Open OnDemand at <https://servood.hpc.ncsu.edu>.
3. **Never benchmark on a login node.** Login nodes are shared and are for editing, light compilation/setup, file transfer, and job submission. All timing experiments must run on allocated compute nodes through Slurm.
4. **Before the prediction freeze, access-checking is allowed.** You may confirm that you can log in, view scheduler availability, prepare/compile code, and submit a trivial non-cache test job. You may **not** run the cache reverse-engineering suite on Hazel, inspect Hazel cache topology/specifications, or collect cache-related PMU counters before the team's prediction is frozen and committed.
5. **Check currently available CPU generations.** Use the official resource page and scheduler tools such as `si` or `sinfo`. The cluster changes over time; the exact CPU model observed inside the allocation is authoritative for your report.

D. Minimal One-Core Slurm Template

Create one job script per target CPU generation. The following is an example; change the constraint and resource limits as appropriate:

```
#!/bin/bash
#SBATCH --job-name=hw1_cache
#SBATCH --output=hw1_%x_%j.out
#SBATCH --error=hw1_%x_%j.err
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=4G
#SBATCH --time=01:00:00
#SBATCH --constraint=haswell

set -euo pipefail
hostname
uname -a
grep -m1 -E 'model name|Hardware|Processor' /proc/cpuinfo

# Run only after the lab-only prediction has been frozen.
srun --cpu-bind=cores ./cache_bench <your arguments>
```

Submit and monitor it with:

```
sbatch hw1_haswell.sh
squeue -u $USER
```

Repeat with each documented CPU-generation constraint that is visible to your course project. The live scheduler output (`si`/`sinfo`) is authoritative. If NC State adds or removes a CPU generation during the assignment, document the live availability and adapt the target list without changing the prediction after the freeze.

Record for every Hazel run: Slurm job ID, requested constraint, hostname, exact CPU model, date/time, logical CPU/core binding, socket/package, NUMA node, compiler and flags, benchmark Git commit, and raw output filename.

E. Required Access Checkpoint

By the early course checkpoint announced by the TA, each team should have: (1) successfully logged in to the assigned ECE lab machines, (2) confirmed Hazel authorization or reported the missing access, (3) cloned/pushed the team GitHub repository and shared

it with both the TA and instructor, and (4) opened the shared Overleaf project and shared it with both the TA and instructor. Access problems reported only near the deadline are not a substitute for completing the experiments.

5 Global Measurement Requirements

These requirements apply to **every reverse-engineering test on every machine** unless the instructor explicitly approves an exception.

- **One million repetitions:** For every tested configuration or x-axis point (working-set size, stride, conflict-set size, latency class, etc.), collect at least **1,000,000 timed samples**. Warm-up samples do not count toward the one million.
- **Distribution, not only an average:** Preserve enough data to reconstruct the latency distribution. Report median, mean, standard deviation, Q1, Q3, 5th/95th percentiles, and the number of outliers. Raw data may be losslessly compressed.
- **Box plots:** For every major test, include box plots of the measured distribution. At minimum, show representative points immediately below, at/near, and above every inferred cache boundary.
- **Sharp-edge evidence:** Capacity, line-size, and associativity plots must use sufficiently dense sampling around the suspected boundary so a convincing transition is visible. A strong result should show a sharp edge at or very near the inferred value. **Do not force or edit data to create an edge;** if the transition is broad, explain why and report uncertainty.
- **Use the highest-resolution trustworthy timer and label its units correctly:** On x86, a serialized `rdtsc/rdtscp` design is appropriate for Phase-I timing. On Arm, use an unprivileged architectural timer if available; otherwise document the highest-resolution timer available. Raw x86 TSC ticks and Arm generic-timer ticks are not automatically instantaneous core clock cycles; the starter-code section explains how to report and validate units.
- **Pin the benchmark:** Bind the benchmark to one logical CPU and record CPU, core, socket/package, and NUMA node. During Phase I, use only topology-identification commands such as those listed in the access section; do not request cache-size fields.
- **Keep memory local on multi-socket systems:** Unless the experiment explicitly studies remote memory, keep the benchmark thread and its memory allocation on the same socket/NUMA domain using first-touch placement and/or non-privileged affinity tools available on the system. Report the locality method. For LLC capacity, report the cache sharing domain visible to the pinned core (normally per socket/package or smaller); do **not** sum LLC capacity across sockets and present it as one cache.
- **Latency units:** Report the raw timer unit/access used in Phase I (for example TSC ticks/access or Arm counter ticks/access) and convert to nanoseconds only when the timer frequency is architecturally/platform documented and trustworthy. Where Phase-II PMU access provides a semantically appropriate CPU-cycle event, use it to validate/report core cycles/access. Do not convert a latency to *cycles* using nominal GHz when turbo/DVFS makes that conversion unreliable. Treat DRAM latency as a system-level quantity influenced by memory technology and NUMA placement, not only by the CPU core.
- **Keep the compiler optimizations turned off by -O0:** Inspect generated assembly for critical loops. Prevent dead-code elimination, unintended vectorization, loop elimination, or reordering that changes the intended access pattern. State compiler, version, optimization flags, and relevant source annotations.
- **Randomize order where appropriate:** Randomize pointer-chase order and, when feasible, randomize the order of tested sizes/strides across trials to reduce drift and prefetcher artifacts. For every major capacity/latency boundary, compare at least one regular/sequential traversal against a randomized dependent traversal as a prefetcher sanity check; use the method least influenced by prefetching for the final inference.
- **Measure latency, not accidental bandwidth:** A cache-latency benchmark must contain a true dependency chain (for example, pointer chasing) so that the next measured load cannot start before the previous one completes. A loop of many independent loads primarily measures throughput/memory-level parallelism and is *not* acceptable as the sole evidence for hit latency.
- **Prefer read-only kernels for the primary cache-latency inference:** Store-to-load forwarding and false load/store address dependencies can add architecture-specific delays unrelated to cache capacity. If you use stores, separate that experiment from the primary load-latency test and explain how you ruled out aliasing/forwarding artifacts.
- **Check cache geometry for self-consistency:** After independently inferring capacity C , line size B , and associativity A , compute the implied number of sets $S = C/(A \times B)$. Report S and use conflict experiments to check whether the inferred geometry is internally consistent. A non-integer or implausible S is a signal to revisit the inference rather than round the answer silently.
- **Record the environment:** Save hostname, CPU model microarchitecture, kernel version, compiler version, affinity, clock/timer method, page size, whether SMT siblings were idle, exact sample count, and any other relevant setting.
- **Report the distribution and do not force LLC latency into one scalar:** Last-level-cache latency can depend on core location, slice/partition, topology, and contention. Report the *distribution* and a representative statistic (e.g., median) plus a range/percentiles when the data show multiple or broad modes.

6 Measurement Terminology and Starter Code

The examples in this section are **starting points, not complete solutions**. You must understand them, inspect the generated assembly, adapt them to the architecture and experiment, and explain why your final measurement is valid. The examples intentionally do not reveal any cache size, associativity, line size, or expected numerical latency for the course machines.

Terminology students are expected to use correctly

Logical CPU.

A scheduler-visible hardware thread number (for example CPU 4 in Linux). With SMT, two logical CPUs may share one physical core.

Physical core.

One processor core. It may expose one or more logical CPUs/hardware threads.

SMT sibling.

Another logical CPU on the same physical core. Activity on the sibling can interfere with execution resources and private caches.

Socket/package.

A physical processor package. Multi-socket systems have separate packages and often separate memory/NUMA domains.

NUMA node.

A memory-locality domain. Access to memory attached to another socket/node can have substantially different latency.

CPU affinity / CPU binding.

Restricting a process or thread to a chosen logical CPU or core so the OS does not migrate it during the benchmark. Migration can change the cache state and timer interpretation.

Working set.

The memory region actively touched by a benchmark during a measurement interval.

Dependency chain.

An access pattern in which the address of load $i + 1$ depends on the value returned by load i . This blocks ordinary memory-level parallelism and makes the loop useful for latency measurement.

Hardware prefetcher.

Processor logic that predicts future memory accesses and fetches data before the demand load. A regular stride can therefore look faster than a randomized dependent traversal.

Compiler barrier.

A directive that prevents the *compiler* from moving memory operations across a point. It does not, by itself, stop out-of-order execution in the processor.

Hardware fence / serialization.

An architectural instruction or sequence used to constrain processor execution ordering around a timing boundary. The correct sequence is ISA-specific.

PMU / performance counter.

Hardware event-counting facilities such as cycles, cache references, misses, or TLB events. These are **for Phase II only** in this homework.

A. Pin the benchmark before collecting timing data

On a Linux lab machine, first inspect topology using the Phase-I-safe topology commands already listed, choose one logical CPU, and launch the program with CPU affinity:

```
taskset -c 4 ./cache_bench <arguments>
```

The number 4 is only an example; use a logical CPU you have chosen and record it. Verify the process is not allowed to migrate outside that set. On Hazel, use the scheduler allocation and CPU binding, for example:

```
srunk --cpu-bind=verbose --cpu-bind=cores ./cache_bench <arguments>
```

If a non-privileged NUMA tool is available and the experiment requires local memory, document the method you use; first-touch allocation after pinning is acceptable when applied carefully. Do not use privileged system changes merely to make a benchmark quieter.

B. Compile with optimizations disabled and inspect the generated assembly

The homework baseline is `-O0`. GCC documents that most optimization passes are disabled at `-O0`. Use a reproducible command such as:

```
gcc -O0 -g -std=c11 -Wall -Wextra -fno-omit-frame-pointer \
-o cache_bench cache_bench.c
```

Do not add link-time optimization (LTO) for the required baseline. Save the exact compiler version and command:

```
gcc --version
gcc -O0 -g -std=c11 -Wall -Wextra -fno-omit-frame-pointer \
-S -o cache_bench.s cache_bench.c
objdump -d ./cache_bench > cache_bench.dis
objdump -d -S ./cache_bench > cache_bench.source.dis
# x86 only, if Intel syntax is easier to read:
```

```
objdump -d -Intel ./cache_bench > cache_bench.intel.dis
```

Inspect the critical loop. Confirm that (1) the intended load actually exists, (2) the dependency chain is intact, (3) the timer reads surround the intended region, (4) the loop was not removed, vectorized, unrolled into an unintended access pattern, or replaced with a library routine, and (5) no unexpected stores are inside the primary read-latency loop. Keep the relevant disassembly excerpt in your experimental notes. Because `-O0` preserves source structure, it also introduces loop/control overhead. Characterize that overhead and use sufficiently long dependent chains or differential measurements so the cache effect is visible above the software overhead. If you also run an optimized diagnostic version, label it separately and do not substitute it silently for the required `-O0` baseline.

A useful compiler-only barrier in GCC/Clang is:

```
static inline void compiler_barrier(void) {
    __asm__ __volatile__("" ::: "memory");
}
```

This tells the compiler not to move memory operations across the barrier. **It is not a CPU fence and is not a substitute for the x86/Arm ordering sequences below.** A `volatile` access can help force an access to remain observable to the compiler, but `volatile` also does not serialize the processor.

C. x86-64 timing example: RDTSC/RDTSCP with ordering

Intel documents that RDTSC is not serializing and describes the use of `LFENCE` (and, when prior stores must also be globally visible, `MFENCE`; `LFENCE`) to order it. On the AMD processors in this assignment, `LFENCE` is dispatch-serializing by default (current microcode with the Linux default), so the same skeleton applies; in any case the ordering the measurement relies on comes from RDTSCP waiting for the prior load to execute, not from `LFENCE` serialization. A simple load-timing skeleton for Intel/AMD x86-64 is:

```
#include <stdint.h>
#include <x86intrin.h>

static inline uint64_t x86_tsc_start(void) {
    _mm_lfence();
    uint64_t t = __rdtsc();
    _mm_lfence();
    return t;
}

static inline uint64_t x86_tsc_stop(void) {
    unsigned aux;
    uint64_t t = __rdtscp(&aux);
    _mm_lfence();
    return t;
}

static inline uint64_t time_one_load(volatile const uint64_t *p) {
    uint64_t t0 = x86_tsc_start();
    uint64_t value = *p;
    uint64_t t1 = x86_tsc_stop();
    __asm__ __volatile__("" : "+r"(value) :: "memory");
    return t1 - t0;
}
```

Measure the empty timing sequence as well so you know the timer/fence overhead distribution. Do not blindly subtract one minimum number from every sample; report how overhead was characterized and verify that your conclusion does not depend on a fragile subtraction.

Important unit note: the x86 time-stamp counter is a time base. On many modern systems it runs at an invariant/reference rate and need not equal the instantaneous core clock under turbo or power management. In Phase I, label the raw quantity accurately as *TSC ticks/access* unless you have justified a stronger interpretation. In Phase II, PMU cycle events can be used to validate a true cycle-based interpretation when permissions and event semantics allow it.

D. AArch64 timing example: Generic Timer

On AArch64 Linux, an unprivileged virtual counter is commonly read through `CNTVCT_ELO`; `CNTFRQ_ELO` reports the nominal counter frequency configured by the platform. Access can be restricted or the frequency can be too low for a useful single-load measurement, so always test it on the actual machine. A conservative starter skeleton is:

```
#include <stdint.h>

static inline uint64_t arm_counter_start(void) {
    uint64_t t;
```

```

__asm__ __volatile__(
    "dsb sy\n\t"
    "isb\n\t"
    "mrs %0, cntvct_el0\n\t"
    "isb\n\t"
    : "=r"(t) :: "memory");
return t;
}

static inline uint64_t arm_counter_stop(void) {
    uint64_t t;
    __asm__ __volatile__(
        "dsb sy\n\t"
        "isb\n\t"
        "mrs %0, cntvct_el0\n\t"
        "isb\n\t"
        : "=r"(t) :: "memory");
    return t;
}

static inline uint64_t arm_counter_freq(void) {
    uint64_t f;
    __asm__ __volatile__("mrs %0, cntfrq_el0" : "=r"(f));
    return f;
}

```

The barrier sequence is intentionally conservative and adds overhead. Characterize that overhead. Because the generic timer may update much more slowly than a core clock, a better Arm method may time a **batch of many dependent pointer-chase loads** between two counter reads and divide by the number of dependent steps. Report *counter ticks/access*; if CNTFRQ_ELO is trustworthy, also report nanoseconds/access as

$$\text{ns/access} = \frac{10^9 (t_1 - t_0)}{f N},$$

where f is the counter frequency and N is the number of dependent accesses. Do not call a generic-timer tick a CPU cycle. If user access to the counter is unavailable, document the highest-resolution unprivileged timer you use instead.

E. Randomized pointer chasing to reduce prefetcher effects

Do **not** attempt to disable system prefetchers with privileged MSR/register writes on shared course machines. Instead, make the access stream difficult for ordinary stride/stream prefetchers and compare it with a regular control. A minimal dependent loop is:

```

struct node {
    struct node *next;
};

static volatile struct node *sink_node;

static void chase(struct node *p, uint64_t steps) {
    for (uint64_t i = 0; i < steps; i++) {
        p = p->next;          /* next address depends on previous load */
    }
    sink_node = p;           /* keep the final result observable */
}

```

A small deterministic shuffle is useful so every run can record and reproduce the same random cycle. For example:

```

#include <stdint.h>
#include <stdlib.h>

static uint32_t xorshift32(uint32_t *state) {
    uint32_t x = *state;
    x ^= x << 13;
    x ^= x >> 17;
    x ^= x << 5;
    return *state = x;
}

static void make_random_cycle(struct node *nodes, size_t n, uint32_t seed) {
    size_t *order = malloc(n * sizeof(*order));

```

```

if (!order || n < 2) exit(1);
for (size_t i = 0; i < n; i++) order[i] = i;

uint32_t state = seed ? seed : 1u;
for (size_t i = n - 1; i > 0; i--) {
    size_t j = (size_t)(xorshift32(&state) % (uint32_t)(i + 1));
    size_t tmp = order[i];
    order[i] = order[j];
    order[j] = tmp;
}
for (size_t i = 0; i < n; i++)
    nodes[order[i]].next = &nodes[order[(i + 1) % n]];
free(order);
}

```

This construction is only a template: choose the allocation size, alignment, node count, and node spacing so they match the experiment, and record the seed. Do not hard-code the cache-line size you are trying to discover.

Before timing, create a set of nodes over the desired footprint, randomly permute their order with a recorded seed, and link them into one cycle. Make node spacing/alignment a benchmark parameter rather than silently assuming the cache-line size you are trying to discover. Warm the chain, then time many dependent steps. For a prefetcher sanity check, run a second chain with regular/sequential order using the same footprint and number of accesses.

What to expect qualitatively: a cache-resident randomized chain should form a relatively low-latency distribution; as the footprint exceeds a cache level, the distribution should shift upward or develop a new mode. A regular traversal may remain deceptively fast at larger footprints because prefetching overlaps future accesses. The final cache-capacity/latency inference should rely on the access pattern least affected by prefetching. Figure 1 shows the kind of distribution separation you should seek, but its numbers are illustrative only.

F. Time a dependency chain, not many independent loads

On every architecture in this assignment, make the batched dependent-chain measurement below your **primary** latency method: time N dependent accesses between a single start/stop pair and divide by N . This is not only an Arm/low-resolution-timer concern. On x86 the LFENCE+RDTSC/RDTSCP overhead is tens of nanoseconds—larger than an L1 hit—so timing one access at a time cannot separate an L1 hit from an L2 hit. Batching amortizes that overhead toward zero and makes the “do not blindly subtract overhead” caveat moot. Keep single-access timing only as a diagnostic. The batched form is:

```

t0 = timer_start();
for (uint64_t i = 0; i < N; i++)
    p = p->next;
t1 = timer_stop();
sink_node = p;
latency_per_access = (double)(t1 - t0) / (double)N;

```

Choose N large enough that timer/fence overhead and counter resolution are small relative to the total interval, while preserving the cache state you intend to measure. The loop is still a latency experiment because every next address depends on the preceding load. By contrast, eight independent array loads per iteration may measure throughput or memory-level parallelism rather than cache-hit latency.

G. Performance counters: Phase II only

Do not run the commands/code in this subsection until the Phase-I timing-only results are frozen. First inspect what the current kernel/PMU exposes:

```

perf list
perf list | grep -Ei 'cache|L1|LLC|TLB|cycle'

```

Then run a representative benchmark with events that actually exist on that machine. Generic Linux names are only examples; their semantics and availability can differ by CPU:

```

perf stat -e cycles,instructions,cache-references,cache-misses \
    -- taskset -c 4 ./cache_bench <arguments>

```

If `perf list` exposes appropriate level-specific events, you may request them, for example L1 data-load/miss or LLC load/miss events. If a command reports unsupported events, permissions errors, or multiplexing, record that fact rather than substituting an event with a different meaning.

The following minimal Linux C example reads a **group of four generic hardware events** around one workload. It demonstrates the mechanism; your final PMU study must use events whose semantics you have checked on the target CPU.

```

#define _GNU_SOURCE
#include <inttypes.h>
#include <linux/perf_event.h>

```

```

#include <stdio.h>
#include <string.h>
#include <sys/ioctl.h>
#include <sys/syscall.h>
#include <unistd.h>

static long perf_event_open(struct perf_event_attr *a,
                           pid_t pid, int cpu,
                           int group_fd, unsigned long flags) {
    return syscall(SYS_perf_event_open, a, pid, cpu, group_fd, flags);
}

static int open_hw(uint64_t config, int group_fd, int leader) {
    struct perf_event_attr pe;
    memset(&pe, 0, sizeof(pe));
    pe.type = PERF_TYPE_HARDWARE;
    pe.size = sizeof(pe);
    pe.config = config;
    pe.disabled = leader;
    pe.exclude_kernel = 1;
    pe.exclude_hv = 1;
    pe.read_format = PERF_FORMAT_GROUP |
                     PERF_FORMAT_TOTAL_TIME_ENABLED |
                     PERF_FORMAT_TOTAL_TIME_RUNNING;
    return (int)perf_event_open(&pe, 0, -1, group_fd, 0);
}

int main(void) {
    int fd[4];
    fd[0] = open_hw(PERF_COUNT_HW_CPU_CYCLES, -1, 1);
    if (fd[0] < 0) { perror("cycles"); return 1; }
    fd[1] = open_hw(PERF_COUNT_HW_INSTRUCTIONS, fd[0], 0);
    fd[2] = open_hw(PERF_COUNT_HW_CACHE_REFERENCES, fd[0], 0);
    fd[3] = open_hw(PERF_COUNT_HW_CACHE_MISSES, fd[0], 0);
    for (int i = 1; i < 4; i++) {
        if (fd[i] < 0) { perror("perf_event_open member"); return 1; }
    }

    ioctl(fd[0], PERF_EVENT_IOC_RESET, PERF_IOC_FLAG_GROUP);
    ioctl(fd[0], PERF_EVENT_IOC_ENABLE, PERF_IOC_FLAG_GROUP);

    /* run the exact benchmark workload here */

    ioctl(fd[0], PERF_EVENT_IOC_DISABLE, PERF_IOC_FLAG_GROUP);

    struct {
        uint64_t nr;
        uint64_t time_enabled;
        uint64_t time_running;
        uint64_t value[4];
    } r = {0};

    if (read(fd[0], &r, sizeof(r)) != (ssize_t)sizeof(r)) { perror("read"); return 1; }
    if (r.nr != 4) { fprintf(stderr, "expected 4 events, got %" PRIu64 "\n", r.nr); return 1; }

    printf("cycles=%" PRIu64 " instructions=%" PRIu64
           " cache_refs=%" PRIu64 " cache_misses=%" PRIu64 "\n",
           r.value[0], r.value[1], r.value[2], r.value[3]);
    printf("time_enabled=%" PRIu64 " time_running=%" PRIu64 "\n",
           r.time_enabled, r.time_running);

    for (int i = 0; i < 4; i++) close(fd[i]);
    return 0;
}

```

If `time_running` is materially smaller than `time_enabled`, the group did not run continuously. For this homework, prefer fewer events

in repeated identical runs rather than allowing multiplexing to obscure a cache transition. Generic events such as `cache-misses` are not automatically L1 misses; for the final eight-event study, use `perf list` and CPU/vendor documentation to select level-specific events and state exactly what each event counts. For derived hit rates, state the exact numerator and denominator and confirm that the events count the same population of accesses.

H. Minimum measurement checklist before trusting a result

For each measured x-axis point: pin/bind the benchmark; warm the intended data structure; verify the timer works; measure timer/barrier overhead; collect the required sample count; retain the distribution; save the seed and exact footprint/stride/conflict parameters; inspect assembly for the critical loop; compare regular and randomized dependent traversal when prefetching is a concern; and record any scheduler, SMT, NUMA, or user interference. Time a batch of *dependent* accesses between one start/stop pair as the primary method and divide by the count; if timer resolution is marginal, increase that count rather than switching to independent loads. On x86 in particular, prefer the batched measurement over timing one access at a time (see Section F), because the fence/timer overhead of a single-access measurement exceeds an L1-hit latency.

Figure 1 is an illustrative example of the distribution evidence expected around cache-residency classes.

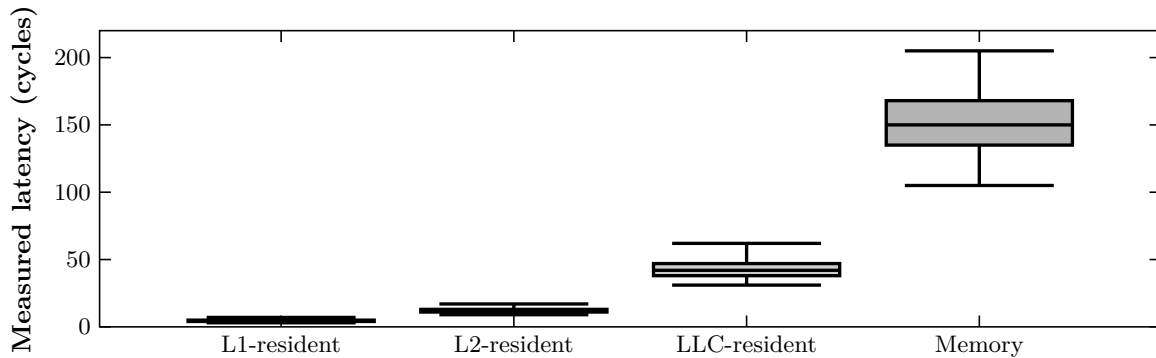


Figure 1: Illustrative latency box plots. One million repetitions should make the distribution, overlap, tails, and outliers visible. Use the actual measured distributions to justify threshold choices and latency classifications.

7 Experimental Rule: Three Separate Phases

The ordering below is mandatory. Save raw results and commit your code/results at the end of each phase before proceeding to the next. Tag the repository checkpoint after Phase I (for example, `phase1-timing-only`) so that the timing-only inference is auditable.

Phase I: Timing-Only Reverse Engineering

During Phase I, infer the cache organization from timing only.

Do not use performance counters or interfaces that directly reveal cache parameters. In particular, do not use `perf stat`, `perf record`, raw PMU events, PAPI, `likwid-perfctr`, or equivalent counter interfaces. Also do not use commands/files whose purpose is to report cache sizes, cache ways, or cache topology (for example, cache entries in `sysfs`) until your timing-only results are frozen.

You may identify the CPU model and ISA so that you know which machine you are using, but do not use published cache specifications to choose or alter your inferred values during Phase I. **In particular, do not consult Agner Fog’s microarchitecture/cache tables, instruction tables, vendor cache tables, or online CPU databases for the machine under test until its timing-only inference is frozen.** These sources contain many of the answers you are being asked to discover experimentally.

Phase II: Counter- and Literature-Based Verification

After Phase I is complete, use available performance counters to test whether the timing-based inferences are correct. You may now inspect counter event lists and system-reported cache information for the processor under test. Keep the timing-only Phase-I values unchanged and clearly separate them from all verification information.

For Intel/AMD x86 lab machines, information for older generations may be available in Agner Fog’s *The Microarchitecture of Intel, AMD, and VIA CPUs* (see the Measurement Methodology References). It is a recommended independent post-Phase-I cross-check because it reports measured cache capacity, ways, sets, line size, latency, and sharing behavior for many generations. Newer CPUs may not yet have equally complete public measurement data.

For each processor, create a compact post-Phase-I validation table modeled on the cache tables in Agner Fog’s *The Microarchitecture of Intel, AMD, and VIA CPUs* (listed in the measurement/background references later in this handout). Table 2 shows the required minimum fields. Keep your measured values and the published/reference values in separate columns.

Level	Measured size	Measured ways	Derived sets	Line	Measured latency	Sharing scope	Reference value	Agreement
L1D								
L2								
LLC								

Table 2: Required post-Phase-I cache validation table for each processor. Keep timing inference, PMU evidence, and published/system information distinguishable in the report.

For the reference column, cite the exact source and section/page. Do not assume that one value applies to every processor carrying the same broad microarchitecture name: Fog’s tables show that L2/LLC sizes and associativities can vary across processor versions/SKUs within a generation. Your measured machine is the unit of analysis. The reference value validates your experiment; it does not replace it.

Phase III: HPC machines

After completing the lab-machine reverse engineering and verification with Performance counters, use the **ECE/lab machines only** (Sunbird, Thunderbird, Skylark, Artemisia, Charnwood, Crux, Ookay, and Upgrade) to build chronological cache-trend models and make predictions for the Hazel HPC CPU generations. You then run the reverse-engineering benchmarks on Hazel and separately inspect Hazel cache specifications/counters.

Repeat the same portable timing-based tools on NC State’s Hazel HPC cluster, where root/sudo privileges are not assumed and PMU access may be restricted. Compare the measured Hazel values against the previously frozen predictions. The prediction exercise remains a required scientific validation task, but the class competition is organized separately into two leaderboards described later: one for reverse-engineering breadth/depth and one for software-only hit-rate estimation accuracy.

8 Exercises (100 points)

Table 3 shows the point redistribution. The Moore-style prediction analysis and the team’s named empirical laws are graded primary results, not unscored add-ons.

Graded component	Points
Architecture and microarchitecture research	8
Timing-only cache reverse engineering	25
PMU verification	10
Eight-counter cross-generation study	8
Software-only cache metric	10
Hazel execution and held-out measurements	10
Moore-style chronological analysis and frozen prediction	12
Two named team cache laws + future scaling-wall analysis	12
Report quality and reproducibility	5
Total	100

Table 3: Homework I grading breakdown.

8.1 Architecture and Microarchitecture Research (8 points)

1. Complete Table 1 for all eight ECE systems. Sort the machines by microarchitecture/processor introduction year, oldest to newest, showing vendor, ISA, microarchitecture, and year. Clearly distinguish Intel, AMD, and Arm, and cite credible vendor or other primary sources for the taxonomy/year fields. (6 pts)
2. Before seeing any cache specification, write two hypotheses about how cache organization or latency might change over time. These are hypotheses, not answers. (2 pts)

8.2 Timing-Only Cache Reverse Engineering — No PMU (25 points)

Write one portable benchmark suite per architecture (or a small set of programs) that determines the cache hierarchy without performance counters. Run it on **all eight ECE machines** unless a machine is temporarily unavailable; document any exception and notify the TA. Use the same benchmark logic and the same one-million-sample requirement on every machine.

Code-listing requirement for this section: As you present each reverse-engineering method in the report, include the corresponding **complete main implementation in the body of the paper**. Show all architecture-specific main implementations actually used (including x86-64 Intel/AMD and AArch64/Arm paths). Do not replace the code with pseudocode or only a GitHub link. If the same parameterized benchmark implements multiple properties, print the complete program once and clearly cross-reference the functions/options used for each property.

For each measurable data-cache level (normally L1D, L2, and LLC/L3), infer the following:

1. **Number of cache levels and effective hierarchy.** Identify the timing plateaus/steps that support your conclusion. Where timing permits, also infer whether a level behaves as private or shared; validate the exact sharing domain only after Phase I. (3 pts)
2. **Cache capacity/size.** Sweep working-set size over a sufficiently wide logarithmic range and identify the transition associated with each level. Use randomized pointer chasing or another method that minimizes prefetching artifacts. (4 pts)
3. **Cache block/line size.** Design a stride or spatial-locality experiment that identifies the smallest transfer/granularity boundary visible in timing. (3 pts)
4. **Associativity (number of ways).** Construct conflict sets by holding the candidate set-index relationship fixed while increasing the number of conflicting lines. Infer the eviction threshold for L1D and L2, and for LLC where practically measurable. If LLC slicing/index hashing prevents a clean measurement, report an effective bound and explain the limitation rather than guessing. (4 pts)
5. **Hit latency of each level.** Measure dependent-load latency so that memory-level parallelism cannot hide the result. A throughput loop of independent loads is not a latency measurement. Report the timer units/access and the full distribution, not only one average; where actual core cycles are validated in Phase II, include them as well. For LLC, report a range/percentiles when topology produces a broad distribution. (4 pts)
6. **Miss/next-level latency.** Measure the transition from an L1 miss to an L2 hit, from an L2 miss to an LLC hit, and from an LLC miss to DRAM when observable. Report both the observed next-level access latency and, where meaningful, the *incremental miss penalty* relative to the previous-level hit. Clearly define every reported number. (4 pts)
7. **Inclusion/exclusion behavior across cache levels.** After you have inferred the hierarchy, sizes, line size, associativity, and latencies, design an experiment that tests whether evicting a line from a lower cache level also invalidates the corresponding copy in an upper level. Use controlled cross-level occupancy/eviction and reload timing to classify the observed behavior as **inclusive**, **exclusive/victim-like**, or **non-inclusive/non-exclusive/uncertain**, as supported by the data. Do not infer policy from capacity ratios alone. Modern LLCs may be hashed, sliced, directory-based, or approximately non-inclusive, so report uncertainty

or a behavioral bound when a clean textbook classification is not supported. Validate against documentation/counters only in Phase II. (3 pts)

Required controls: pin execution to one CPU/core; warm up; repeat enough times to report median and variability; avoid compiler elimination; serialize timing where needed; use dependent/randomized accesses to suppress prefetching when appropriate; compare regular vs. randomized traversal as a prefetcher check; keep the primary latency kernel load-only unless a store is essential; report whether SMT siblings or other users may have interfered.

Required plots: Figure 9 and Figure 8 later in the handout show only the expected *shape* of convincing boundary evidence; they are not numerical answers.

- latency vs. working-set size,
- latency vs. stride,
- latency/eviction probability vs. number of conflicting addresses,
- **cross-level eviction/reload evidence for inclusion/exclusion behavior**, showing the condition that distinguishes whether an upper-level copy survives or is invalidated after lower-level eviction pressure,
- a final inferred cache table for every machine.

Each final table must contain at least:

{level, size, line size, associativity, derived sets, hit latency, miss/next-level latency, sharing scope, inclusion/exclusion behavior}.

Your report must explain your own reasoning and show why the measured curve implies the claimed structure.

Cache levels and capacity.

Build a dependent randomized pointer-chase so the next address cannot be issued until the previous load completes. Sweep working-set size from much smaller than expected L1 to much larger than LLC, initially by powers of two and then densely around each observed transition. Plateaus correspond to different residency levels; a jump suggests the working set no longer fits in the previous level. Run a regular/sequential version as a control: if it is much faster than the randomized chain at larger footprints, hardware prefetching is probably helping and should not be mistaken for a larger cache.

Line/block size.

Keep the total footprint controlled and sweep byte stride or offset between accesses. Look for the spatial-locality granularity at which neighboring bytes stop benefiting from the same fetched line. Repeat densely around candidate powers of two such as 32, 64, 128, and 256 bytes. Distinguish a line-size effect from prefetching by using randomized line order, and explicitly test accesses just below/at/above the suspected line boundary. Record alignment so that a boundary-crossing effect is not confused with line size.

Associativity.

Create groups of addresses that are likely to map to the same cache set while varying the number of simultaneously live lines. A sudden increase in eviction probability/latency after adding one more congruent line is evidence for the number of ways. Start by controlling low address bits/page offset, then search for mutually conflicting addresses empirically. Modern LLC hashing/slicing can make the LLC result an *effective* associativity rather than a simple textbook value. After finding C , B , and A , compute $S = C/(A \times B)$ and use the implied set count as a cross-check against your conflict-stride behavior.

Hit latency.

Warm a known line, serialize the timing region, repeatedly access it through a true dependency, and subtract/characterize timer + loop overhead. Keep a separate box plot for the hit distribution. As a diagnostic, also measure a version with independent loads: it may have much better cycles/load because it exposes bandwidth/parallelism, which is precisely why it must not be used as the latency number.

Next-level and memory latency.

Force the target line out of the desired level using a controlled eviction/working-set construction, then measure the first dependent access. Compare distributions rather than a single number; overlapping modes often reveal intermediate levels or interference. Especially for LLC, a wide range may be real because access time can depend on where the cache line resides relative to the requesting core.

Hierarchy confirmation.

Cluster or visually compare latency modes and correlate them with the sharp working-set boundaries. A valid conclusion should be supported by both the size sweep and a latency-distribution change.

Inclusion/exclusion behavior.

First establish two cache levels and a target line whose upper-level residency you can test. Create lower-level eviction pressure with addresses selected to displace the target from the lower level while avoiding a direct intentional eviction from the upper level when possible. Reload the target and compare its latency/distribution with calibrated upper-level-hit and deeper-level-hit classes. If lower-level eviction consistently removes the upper-level copy, that is evidence of inclusive behavior; if the upper-level copy survives or lines migrate between levels, the behavior may be exclusive/victim-like or non-inclusive. Repeat with controls and multiple target sets/addresses. State exactly what behavior your experiment proves; do not claim a global policy from one ambiguous case.

Load/store aliasing confound.

Agner Fog documents architecture-specific false memory dependencies and store-forwarding/address-alias effects for multiple Intel and AMD generations in *The Microarchitecture of Intel, AMD, and VIA CPUs*; see the Agner Fog entry in the Measurement Methodology References and the CPU-specific sections relevant to the processor under test. Therefore, do not mix a store stream into your primary capacity/latency pointer chase unless you intentionally study that effect. If stores are present, vary the address spacing and demonstrate that the observed cache boundary is unchanged.

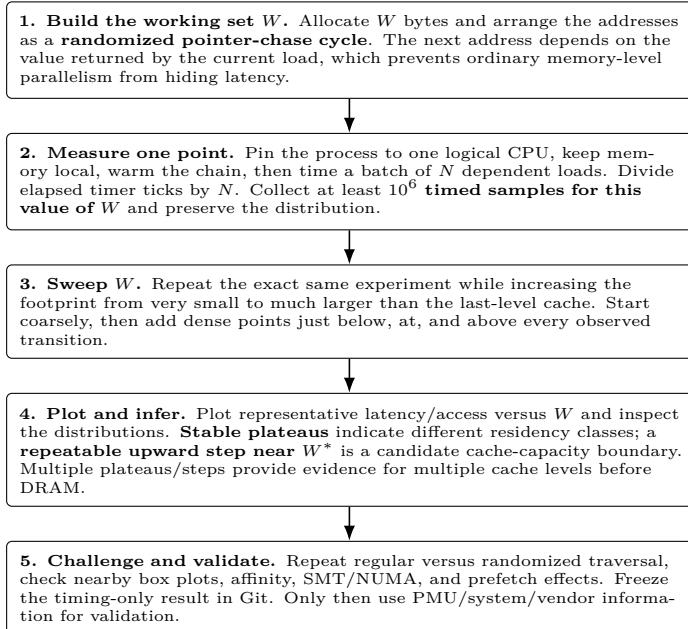
Software-only hit rate.

Calibrate timing distributions for strongly cache-resident and strongly non-resident accesses. Use a threshold, likelihood model, or mixture model to classify new accesses. Validate against PMU misses only in Phase II.

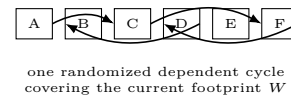
Example reverse-engineering diagrams: show what the benchmark actually does

The methodology diagrams in the report and slides must be **concrete experimental diagrams**, not generic boxes that only say “hypothesis → measure → infer.” A reader should be able to see *what addresses are accessed, what parameter is changed, what is timed, what curve is produced, and what feature of that curve becomes the inference*. Figures 2 and 3 illustrate the expected level of detail. They are schematic and intentionally contain no numerical answer for the course machines.

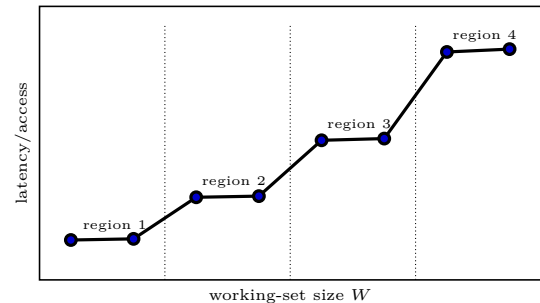
Example A: how to produce a cache-level/capacity plot. A plot with several latency regions/plateaus, like the attached/class example for “determine the number of cache levels,” should be explained with the experimental chain below.



What the benchmark is doing



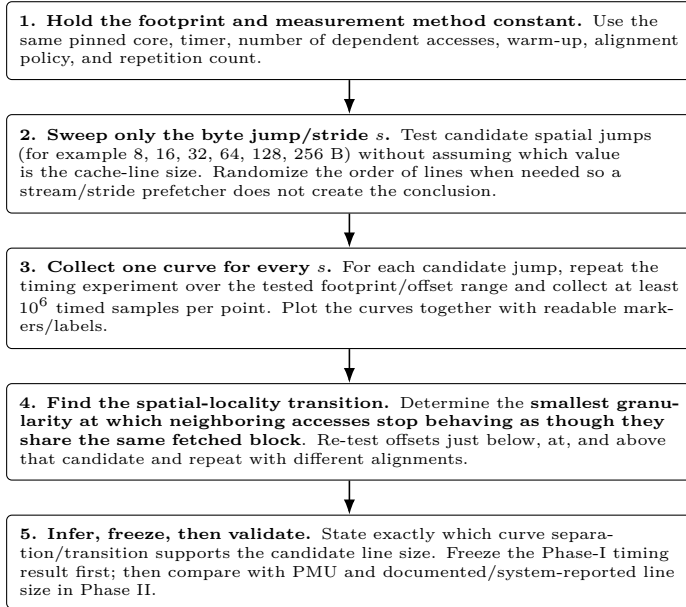
How those measurements become the result



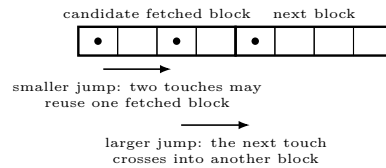
Inference example: count repeatable latency regions and identify the working-set sizes where the transitions occur. The exact numerical locations must come from the team’s data, not from this schematic.

Figure 2: Example of the required reverse-engineering logic for a cache-level/capacity result. The diagram shows the actual address construction and measurements that produce each point on a plateau/transition plot, followed by the inference rule.

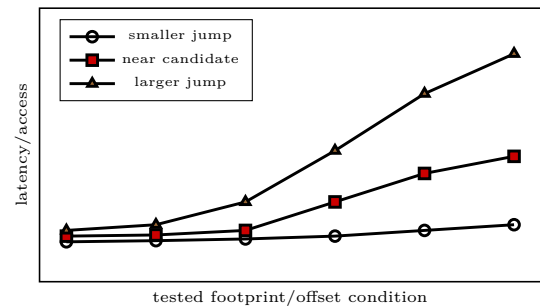
Example B: how to produce a cache-line-size result. For a result that overlays several byte jumps/strides, like the attached/class line-size plot, show how the different curves were generated instead of presenting only the final figure.



What changing the jump means



How the family of curves is interpreted



Inference example: identify the first repeatable change in spatial behavior, then verify it with just-below/at/just-above offsets and alignment controls. Do not infer the line size from one noisy crossing or one prefetch-sensitive curve.

Figure 3: Example of the required reverse-engineering logic for a cache-line-size result. The diagram shows how candidate byte jumps create the measured family of curves and how the spatial-locality transition becomes the line-size inference.

Required slide format for every reverse-engineering result. Each result slide should contain four things together: (1) a small access-pattern/experiment diagram such as the examples above, (2) the numbered experimental steps, (3) the team's actual measured evidence plot, and (4) a one- or two-sentence finding stating exactly what feature of the plot supports the inferred value or behavior. For example, the cache-level slide should explicitly connect its repeatable plateaus/transitions to the inferred residency classes/boundaries; the line-size slide should explicitly identify which change among the candidate jump experiments supplies the line-size evidence.

8.3 Verify the Reverse Engineering with Performance Counters (10 points)

Only after freezing the Phase-I results, repeat representative capacity, line/stride, and associativity experiments while collecting cache-related performance events. Preserve the same workload and, for each verified point, collect one million workload repetitions so the timing distributions remain directly comparable with Phase I. Counter collection may be split into identical runs if multiplexing eight events would distort the measurement.

1. Run `perf list` (or the platform-equivalent event list) and identify the cache events that are actually exposed on each machine. Do not assume event names are identical across Intel, AMD, and Arm. (2 pts)

2. Use counters to locate the same cache transitions observed in timing. For example, show where an inferred L1 capacity boundary coincides with increasing L1 misses, or where an LLC boundary coincides with increasing last-level misses. (3 pts)
3. Compare the timing-only inference against system/vendor-reported cache organization after Phase I is complete. For Intel/AMD lab systems, include Agner Fog as one reference when that processor generation is covered. Create a table with **Timing Inference**, **PMU Evidence**, **Published/System Value**, **Source/Page**, and **Error/Agreement**. Validate size, ways, derived sets, line size, latency, sharing scope, and inclusion/exclusion behavior where suitable evidence or documentation is available. (3 pts)
4. Analyze disagreements. (2 pts)

8.4 Eight Interesting Performance Counters Across Generations (8 points)

Choose exactly **eight** performance events that help explain cache/memory behavior. Prefer semantically comparable generic events across machines, and use the closest architecture-specific equivalent only when necessary.

A useful starting set to investigate is:

- cache references,
- cache misses,
- L1 data-cache accesses/loads,
- L1 data-cache misses,
- L1 data-cache stores or replacements,
- LLC accesses/loads,
- LLC misses,
- data-TLB misses or another memory-hierarchy event that you can justify.

This list is not a requirement to use those exact names; event availability and semantics differ by PMU. Record the exact event name and its meaning on every architecture.

1. Run the **same three standardized microbenchmarks** on every ECE machine: (i) L1-resident dependent accesses, (ii) LLC-sized randomized accesses, and (iii) a working set larger than LLC. Collect all eight events. (3 pts)
2. Normalize raw counts where appropriate (e.g., per access, per instruction, or per 1,000 iterations). Explain the denominator. (1 pt)
3. For each benchmark, rank/sort the machines by each normalized event or derived rate. Present an S-curve/ranked summary and compact tables rather than eight disconnected pages of plots. (2 pts)
4. Compare Intel vs. AMD vs. Arm and compare older vs. newer generations. Discuss which counters remain comparable, which do not, and why raw event counts can be misleading across different PMUs. (2 pts)

8.5 Software-Only Replacement for a Cache Metric (10 points)

Assume that a machine does **not** permit your process to read hardware cache counters. Write a software-only microbenchmark that estimates a cache metric from timing alone.

The required metric is a **timing-derived cache hit rate** (or a closely related cache-residency metric approved by the instructor). This estimator is also the basis of Competition, so design it for portability and generalization rather than tuning it to one machine:

$$\hat{H} = \frac{N_{\text{timed accesses classified as cache hits}}}{N_{\text{timed accesses}}}.$$

1. Design a calibration procedure that produces timing distributions for known/strongly expected cache-resident accesses and known/strongly expected non-resident accesses. (3 pts)
2. Define and justify a classification rule or probabilistic estimator. A fixed threshold is allowed only if the timing distributions support it. (2 pts)
3. Implement the estimator without PMU access and report confidence/uncertainty. (2 pts)
4. On ECE machines where counters are accessible, compare your software estimate with a PMU-derived hit/miss rate. Quantify absolute and relative error. Explain when this estimator fails. (3 pts)

8.6 NC State Hazel HPC (10 points)

NC State's Hazel cluster is heterogeneous and currently exposes multiple CPU generations through Slurm architecture constraints. Unlike the ECE lab machines, **do not assume sudo/root access or unrestricted PMU access on Hazel**. Your primary tools on Hazel must therefore be the timing-only reverse-engineering code and the software-only cache metric from the previous sections.

Do not run benchmarks on HPC login nodes. Connect through `login.hpc.ncsu.edu`, then submit to compute nodes through Slurm. NC State documents the CPU-generation constraints summarized in Table 4. **Availability to the course project and to Slurm can change**, so check `si/sinfo` and the official Slurm deployment-status/resource pages immediately before each run.

Slurm constraint	CPU family/model listed by NC State	Generation to research
haswell	Intel Xeon E5 v3	Intel Haswell
broadwell	Intel Xeon E5 v4	Intel Broadwell
skylake	Intel Xeon Scalable Processor	Intel Skylake-SP
cascadelake	Intel Xeon 2nd Gen SP	Intel Cascade Lake
icelake_6326	Intel Xeon Gold 6326	Intel Ice Lake-SP
icelake_8358	Intel Xeon Platinum 8358	Intel Ice Lake-SP
sapphirerapids	Intel Xeon 4th Gen	Intel Sapphire Rapids
genoa	AMD EPYC 4th Gen	AMD Zen 4 / Genoa
turin	AMD EPYC 5th Gen (when visible in Slurm)	AMD Zen 5 / Turin

Table 4: Hazel Slurm constraints and CPU generations to attempt. Scheduler visibility to the course project is authoritative and may change during the assignment.

The constraints in Table 4 must be checked against live scheduler visibility. Official references: <https://hpc.ncsu.edu/RunningJobs/Resources.php> and <https://hpc.ncsu.edu/RunningJobs/Status.php>. Hazel is dynamic; record the exact CPU model returned by the allocated node rather than assuming that every node with a constraint is identical.

Target coverage. Attempt to run on **every distinct CPU generation that is currently accessible to the course project in Slurm**. At minimum, complete successful experiments on **five distinct CPU generations**, spanning the oldest and newest accessible generations and including an AMD generation (Genoa or newer) if accessible. If a documented generation is not visible or remains unavailable after reasonable attempts, document the scheduler availability/job request and continue with the accessible set.

1. **Verify the prediction freeze before any Hazel cache run.** The scientific model/prediction is graded in the Moore-style section. Here, record the already-frozen prediction table, Git commit hash, and timestamp and demonstrate that the freeze predates the first Hazel cache experiment. (1 pt)
2. For each selected Hazel generation, submit a Slurm compute job using the appropriate **-constraint**, bind the benchmark to a core, and record hostname, exact CPU model, logical CPU, physical core, socket/package, NUMA node, and job ID. Include the job scripts. (2 pts)
3. Run the same Phase-I cache suite with **1,000,000 samples per tested point** and infer cache hierarchy, capacity, line size, associativity/effective associativity, hit latency, next-level/memory latency, and inclusion/exclusion behavior. Include the same curve + box-plot evidence used on the lab machines. These are the held-out observations used to evaluate your prediction. (4 pts)
4. On each prediction plot, overlay the Hazel measurements using a visually distinct marker (for example a star or diamond) while preserving the original dashed prediction. Discuss prediction accuracy for every predicted quantity and identify where the model was closest, where it failed, and why. (1 pt)
5. Only after the Hazel timing-only values have been frozen, research each Hazel target’s published cache organization and compare it with both your prediction and your timing-only reverse engineering. (1 pt)
6. Run your software-only cache-hit/residency estimator and combine Hazel and ECE results into one chronological cross-architecture dataset. (1 pt)

9 Moore-Style Cross-Generation Results and Prediction Analysis (12 points)

This is a **required primary result of the homework**, not an optional discussion.

Grading for this section (12 points):

1. Chronological master table plus the required cross-generation plot set, with correct year convention, units, labels, uncertainty, and cross-architecture visual encoding. (3 pts)
2. Quantitative trend analysis: fit only defensible models, report doubling time/slope/step frequency as appropriate, and discuss constant or non-monotonic metrics honestly. (2 pts)
3. **Frozen lab-only prediction:** using only ECE/lab observations, construct the quantitative Hazel prediction table/curves, show the dashed extrapolation, state uncertainty, and freeze the model/parameters/predictions in Git before any Hazel cache experiment. (3 pts)
4. **Held-out evaluation:** overlay the later Hazel observations without refitting the frozen dashed model and quantify which predictions succeeded or failed and why. (2 pts)
5. Make the required additional quantitative prediction about five years beyond the newest measured system and explain the model’s plausible failure modes. (2 pts)

The chronological analysis has two visually and methodologically separate roles:

1. **Lab machines = observations/training data.** Use Sunbird, Thunderbird, Skylark, Artemisia, Charnwood, Crux, Ookay, and Upgrade to establish the observed trend and make the frozen prediction.

2. **Hazel machines = held-out validation data.** Overlay Hazel only after the predictions are frozen and compare whether the previously predicted trend was correct.

After validation, combine all successfully measured systems into one chronological dataset for the final historical comparison. Sort the rows and plot points by the **introduction year of the processor generation/microarchitecture**, from oldest to newest, in the same empirical spirit as Moore’s original plot.

Required chronological master table

Create one master table with **one row per measured machine/CPU generation**, sorted oldest to newest. Table 5 shows the required identifying columns; append the cache columns described below.

Year	Vendor / ISA	Microarchitecture	Machine / CPU	Process node
Then append all measured cache columns described below.				

Table 5: Required chronological master-table skeleton. Append the timing-only cache columns and optional verification columns, then sort the final rows by the stated year convention.

The cache columns must contain the timing-only inferred value, and you may add a second value or verification flag for PMU/published agreement. Do not silently replace an inferred result with a vendor specification.

Required Moore-style plots

Figures 4, 5, 6, and 7 illustrate the required observed-solid / predicted-dashed / held-out-reveal visual logic. Produce a chronological plot for **each major quantity you reverse engineered**. Every plot must show all architectures on the same axes when the quantity is meaningfully comparable. Use a consistent visual encoding across the entire report (for example, distinct marker shapes for Intel x86, AMD x86, and Arm/AArch64) and label points with the microarchitecture or CPU generation when readability permits.

At minimum, include the following plots:

1. **L1D capacity vs. year.**
2. **L1D associativity vs. year.**
3. **L1D hit latency vs. year.** Report validated core cycles/access where a suitable cycle measure is available; for cross-architecture comparison also report nanoseconds/access from a documented/calibrated timer when possible. Never manufacture “cycles from nominal GHz.
4. **L1 miss penalty vs. year.** Define the penalty precisely, e.g., the incremental cost of falling through L1 to the next level or the measured L2-resident access latency.
5. **L2 capacity per core vs. year.**
6. **L2 associativity vs. year.**
7. **L2 hit latency vs. year.** Use the same unit discipline as L1D: validated core cycles/access where available, and a comparable time unit (preferably ns/access) for cross-architecture comparison.
8. **L2 miss penalty vs. year**, with a precise definition of the next level being reached.
9. **LLC/L3 capacity vs. year.** Show the capacity of the **LLC sharing domain visible to the pinned core** (for example, per socket/package, CCD/CCX, or other documented domain after Phase I), and also a normalized value such as MiB/core when meaningful. Do not sum independent sockets into one cache value.
10. **LLC associativity/effective associativity vs. year** where it can be measured reliably; mark bounds or uncertain values explicitly.
11. **LLC hit latency and LLC-to-memory miss penalty vs. year.** State whether the plotted unit is validated core cycles/access, timer ticks/access, or ns/access, and use comparable units when combining architectures.
12. **Cache line/block size vs. year.**
13. **Inclusion/exclusion behavior vs. year.** Because this is categorical, use a clearly labeled categorical/step-style summary (for example inclusive, exclusive/victim-like, non-inclusive/non-exclusive, or uncertain) rather than forcing a numeric trend.
14. **Your timing-derived hit-rate/cache-residency metric vs. year** under one identical standardized workload.
15. **At least two of the eight PMU-derived normalized metrics vs. year**, using only events whose semantics are sufficiently comparable across the plotted machines.

Plot format. For cache capacities, make the y-axis logarithmic or plot $\log_2(\text{bytes})$, so that a doubling appears as an equal vertical step, similar to Moore’s component-count diagram. For latency/penalty and associativity, use the scale that best represents the data without hiding differences. Put the actual year on the x-axis, not an arbitrary machine index. Show measured points even if the trend is flat or irregular. Include uncertainty/error bars where the inferred value or latency has measurable variability. For held-out prediction figures,

connect the examined/collected lab observations with a **solid line and solid markers**; begin the **dashed frozen extrapolation at the last measured lab point and extend it only into future/unmeasured years**; use a **different marker such as a star/diamond for later Hazel measurements**. The predictive trend must be fit from at least **three lab observations**; Hazel points must never be used to refit the dashed line in the held-out validation figure.

Required figure style (conference/old-ISCA style). All result plots in the report and slides should look like classic publication-quality gnuplot figures: **large readable fonts**, white background, black/grayscale lines and markers, clearly distinguishable marker shapes and line styles, no background grid lines, strong black axes, and line widths/marker sizes that remain readable when the figure is reduced to a paper column. Avoid rainbow palettes, thin hairline curves, tiny tick labels, decorative backgrounds, and legends that obscure data. Use solid markers and solid connecting lines for examined/collected observations. Use dashed lines for future-year prediction/extrapolation beyond the last measured training-year point; do not draw the prediction as a dashed overlay across already measured years. Figures must remain interpretable when printed in grayscale.

Cross-architecture comparison. Intel, AMD, and Arm points must be visible together wherever the metric is semantically comparable. Do not connect unrelated architectural families with a line that implies direct lineage. You may additionally show separate trend lines for Intel, AMD, and Arm if enough points exist. If a quantity is not directly comparable across ISAs/PMUs, state why and either normalize it carefully or present separate panels while preserving the same year axis.

Minimum evidence for a fitted trend: a fitted/predictive series must contain at least three lab-machine observations. A vendor-specific trend line requires at least three observations from that vendor/family; otherwise, do not imply a vendor-specific law. If you use a pooled cross-architecture model to predict a family with sparse lab data (for example AMD), label it explicitly as a pooled model and discuss that limitation.

9.1 Your Team’s Cache Laws and Future Scaling Walls (12 points; mandatory: at least two laws)

In the spirit of Moore’s empirical observation, each two-person team must formulate at least **two quantitative empirical “laws”** that summarize trends discovered from the **ECE lab machines only**. Name each law using the two team members’ last names. For example, a team consisting of Student Smith and Student Jones might propose:

Smith–Jones Cache Capacity Law and Smith–Jones Cache Latency Law.

The exact names and quantities are yours to choose. At least two laws must concern **different cache-related quantities**; for example, cache capacity, hit latency, miss penalty, associativity, line size, LLC capacity per core, or another defensible derived cache metric. One law may describe growth while another may describe a latency cost, a stepwise trend, or even a quantity that remains approximately constant.

Grading for the two-law assignment (12 points total): quantitative law formulation and evidence (4 pts); frozen law-specific Hazel predictions and held-out tests (3 pts); limitations/scope discipline (1 pt); and the future scaling-wall analysis below (4 pts).

Each proposed law must include all of the following:

1. **Name.** A short memorable name containing both teammates’ last names, followed by “Law.”
2. **One-sentence statement.** State exactly what quantity changes with processor/microarchitecture year and in what way. Avoid vague claims such as “caches get better.”
3. **Quantitative rule/model.** Give the fitted equation, doubling time, slope, step frequency, bounded/constant rule, or other quantitative relationship supported by your lab observations.
4. **Scope/domain.** State which observations the law is based on: years, machines, vendors/ISAs, and whether it is vendor-specific or pooled across architectures. Do not imply universality outside the measured domain.
5. **Evidence plot.** Show at least three lab-machine observations as solid points/solid line and the law’s extrapolation as a dashed line, in the Moore-style format used throughout this assignment.
6. **Frozen prediction.** Before running the Hazel cache experiments, use the law to predict the corresponding quantity for one or more held-out Hazel generations. Freeze the law, its parameters, and its predictions in GitHub together with the other held-out predictions.
7. **Held-out test.** After the freeze, measure Hazel and place the Hazel result on the same figure using the designated star/diamond marker. Explain whether the held-out data **supports, weakens, or falsifies** the proposed law and why.
8. **Limitations.** Identify at least one plausible reason the law may fail in another generation or architecture (for example design-goal changes, process technology, chiplet organization, memory technology, server/desktop class, or vendor-specific policy).
9. **Future scaling wall / limit analysis (4 pts across the two laws).** For **each of your two law metrics**, ask whether the measured trend could continue indefinitely. Identify the most plausible hard or soft limit and explain the microarchitectural reason. For a capacity/scaling law, consider factors such as SRAM density and access time, die/chiplet area, leakage and dynamic energy, wire/interconnect delay, banking/slicing, coherence traffic, and the sharing domain. For a latency/cost law, consider SRAM and wire delay, physical distance, clock period/frequency, pipeline organization, queueing/contention, topology, and the distinction between reducing true access latency and merely hiding latency with speculation or memory-level parallelism. Distinguish a *physics/technology limit* from a *design-choice/economic limit*. If your data support a quantitative bound, asymptote, turning point, or “wall” estimate, show it with uncertainty; if they do not, state what additional evidence would be needed rather than inventing

an exact year/value. Conclude whether each law is likely to continue, flatten, reverse, or become piecewise in future generations and why.

At least one of the two laws should address **capacity/scaling** (for example L1, L2, LLC, or normalized cache capacity) and at least one should address a **cost or behavior** (for example hit latency, miss penalty, or another cache-related performance cost), unless your measurements provide a strong technical reason for a different pair.

Required interpretation and prediction

For every major plot, write a short interpretation identifying the strongest chronological pattern (or lack of one). Then answer:

1. Which cache quantities are approximately constant over time (for example, a line size may remain unchanged for many generations)?
2. Which quantities grow with time? Is the growth closer to linear, stepwise, or exponential/doubling behavior?
3. Which latencies or miss penalties increase even when capacity increases? Discuss the capacity–latency tradeoff where visible. For DRAM/memory penalty, separate CPU-generation effects from platform effects such as memory technology, NUMA locality, and server configuration.
4. Are the trends different for Intel, AMD, and Arm? Are they better explained by vendor, ISA, server/desktop class, or microarchitecture generation?
5. Which measured quantity most resembles the style of trend Moore observed? Fit a simple model only when defensible and explain how well the model agrees with the observations rather than judging only by eye.
6. For any approximately exponential capacity trend, estimate a **doubling time**. For other trends, report an appropriate slope or step frequency.
7. First evaluate the **lab-only prediction against the held-out Hazel measurements**: which predicted cache quantities were accurate, which failed, and why? Then make at least **one further quantitative prediction** for a CPU generation approximately five years beyond the newest measured system. Present it with at least three observed points and a dashed projected continuation. State the predicted value, uncertainty range, model used, and reasons the prediction could fail.
8. Compare the strength of your evidence with Moore’s original empirical observation. Your data may show that some cache properties scale while others are deliberately held nearly constant. Do not claim a “law” unless the evidence supports it.

Slides requirement: The presentation must contain a compact “cache evolution” sequence: at least one Moore-style capacity plot, one Moore-style latency/penalty plot, and one cross-architecture comparison plot. **The slides must also show an actual evidence plot for every reverse-engineering inference** (cache levels/hierarchy, capacity, line size, associativity, hit latency, miss/next-level latency, and inclusion/exclusion behavior); multiple inference plots may share one slide when readable. Do not present only the final inferred number without the plot that supports it. At least one slide must show the held-out validation logic explicitly: **lab observations** → **dashed frozen prediction** → **Hazel reveal**, together with a concise discussion of how accurate the frozen prediction was. Include a dedicated “**Our Laws**” slide naming both team laws, stating each quantitative rule in one sentence/equation, and showing whether the held-out Hazel evidence supports, weakens, or falsifies it. A small-multiple summary slide showing L1, L2, and LLC evolution together is strongly encouraged.

10 Report Quality (5 points)

Treat the report as a short architecture research paper rather than a lab worksheet. The 5 report-quality points reward clarity, traceability, correct figure/table references, captions/units, and reproducibility rather than decorative formatting. Use the following title:

Predicting Cache Evolution: The Growing Capacity and Latency Cost of CPU Caching

The paper should center on whether historical measurements from the lab machines reveal trends in cache capacity and latency that can be used to predict later CPU generations, whether those frozen predictions are supported by the held-out Hazel measurements, and whether the team’s two named empirical cache laws survive the held-out test. A well-structured report must clearly separate **timing-only inference**, **counter-based verification**, and **published/system information**. Every figure/table must have a caption, units, machine identification, and a short interpretation. Include raw data and scripts sufficient for reproduction.

Required code in the body of the paper. Do not hide the implementation only in GitHub, the Moodle archive, or an appendix. In the main body, each reverse-engineering methodology section must show the **main code that produced the result**. Include complete main listings for every distinct reverse-engineering implementation and every architecture-specific path used in the experiments. At minimum, the reader must be able to see the code that constructs the access pattern, controls the swept parameter, performs timing/serialization, records samples, and implements the inference-specific eviction/conflict logic. Label each listing with the target property and architecture (for example, x86-64 or AArch64), and state the exact source filename/Git commit that the listing corresponds to. Reused common code does not need to be printed repeatedly, but it must appear in full at least once in the body and later sections must point to the relevant listing. Plot-only utilities and generic data-format helpers may remain in the ZIP/GitHub unless they are necessary to understand how a reported measurement was generated.

For each major reverse-engineering target, organize the methodology/results so the reader can follow the full chain: **code/access pattern** → **controlled sweep** → **measured distribution/plot** → **inference rule** → **Phase-II validation**. The code listing and the concrete reverse-engineering diagram should agree with the actual experiment that generated the plotted data.

The report must also contain a **methodology-logic figure set**: for each major reverse-engineering target (cache-level discovery, capacity, line size, associativity, hit latency, miss/next-level latency, inclusion/exclusion behavior, and the software-only cache metric), include a small diagram or flow chart showing the logic of the experiment, the measured quantity, the expected transition/signature, and the inference rule. These figures should help a reader understand not only *what* you measured, but *why* the result reveals the cache property.

Slides: Prepare a concise presentation (approximately 10–15 content slides) covering the experimental question, machine timeline, one flow chart per reverse-engineering logic, PMU validation, eight-counter comparison, the software-only hit-rate metric, Hazel replication, and the Moore-style chronological results. **For every reverse-engineering inference, the slides must include the corresponding measured plot/evidence figure** (for example, capacity edge, stride/line-size signature, conflict/associativity threshold, latency distribution, miss/next-level transition, and inclusion/exclusion cross-level eviction/reload evidence). On the slide for each reverse-engineering method, include a short numbered or bulleted **step-by-step itemization** stating: (1) what is varied, (2) what is held constant/controlled, (3) what is measured, (4) what signature or transition is sought, (5) how that signature produces the inferred property, and (6) how the inference is validated later. The slides must visibly compare L1, L2, and LLC size/associativity/latency or penalty across generations and across Intel, AMD, and Arm, followed by the prediction, limitations, and contributions.

11 Suggested Experimental Practices

- Prefer dependent pointer chasing when measuring latency; independent accesses can measure throughput instead of latency. Include one independent-load diagnostic to demonstrate that you understand the distinction.
- Use randomized traversal orders to reduce stride-prefetcher effects. Compare a regular traversal with the randomized traversal at representative footprints and discuss any prefetching benefit you observe.
- Keep primary latency/capacity kernels read-only when possible. Mixed stores and loads can trigger store-forwarding or address-alias dependencies that masquerade as cache effects.
- Record alignment and page-boundary crossings. Misaligned accesses and TLB/page effects can create steps that are not cache-capacity boundaries.
- Sweep sizes densely around suspected boundaries after a coarse logarithmic sweep.
- Report medians and distributions/percentiles; a single minimum or mean is not sufficient.
- Use **CPU affinity**, meaning that the operating system is instructed to keep the benchmark on a chosen logical CPU or chosen set of CPUs instead of freely migrating it. Migration can change which private cache is used and can corrupt a latency measurement. On a lab machine, a typical command is `taskset -c <logical-cpu> ./cache_bench ...`. On Slurm, use CPU binding (for example, `srun -cpu-bind=cores`) and record the allocation. See the terminology and starter-code section for the distinction between a logical CPU, physical core, SMT sibling, socket/package, and NUMA node.
- If you cannot obtain exclusive access, repeat enough times to quantify interference and explicitly mark noisy measurements.
- Avoid assuming that the LLC is a single simple set-associative structure. Modern processors may use slices, hashes, non-inclusive policies, victim structures, and complex replacement policies.
- Counter events may be multiplexed when too many are requested simultaneously. If necessary, collect them in multiple identical runs and report the procedure.

12 Data Organization and Traceability

Submit the complete reproduction path, not only the data. For every machine and experiment, preserve the exact source code or Git commit used, the build command, compiler and version, all compiler/linker flags, the run command and command-line inputs, affinity/binding command, Slurm script when applicable, environment variables that affect the experiment, random seed, sample count, and the raw output filenames. A grader should be able to move from a machine/experiment folder to the exact command that generated its data without guessing.

Main-code snapshot in the Moodle ZIP. In addition to GitHub history, place the final main source files used for the submitted results directly inside the Moodle ZIP. Organize them by architecture and purpose so a grader does not have to reconstruct the implementation from commit history. The archive copy must match the code listings printed in the body of the report and the Git commit recorded for the final runs.

Submit all data that you collect, not only the points that appear in the final paper. The graders must be able to start from the raw outputs and reproduce every reported statistic, box plot, cache-boundary curve, PMU table, chronological trend, and prediction figure.

Organize the data **first by machine/server**, then by experiment. Each ECE lab machine and each Hazel CPU generation that you test must have its own folder. Within that folder, keep separate subfolders (or clearly named files) for capacity/hierarchy, line size, associativity, latency, PMU validation, the software-only cache metric, and any other experiment you run. Preserve the complete one-million-sample data or a losslessly compressed representation sufficient to reconstruct the full distribution.

For example:

```

machines/
  sunbird/
    README.md          # exact build/run commands + environment
    main_code/         # final source snapshot matching report/Git
      x86_64/          # Intel/AMD main reverse-engineering code
      aarch64/         # Arm main reverse-engineering code
      common/          # shared portable code, if used
    build/             # compiler/version/flags and build log
    capacity/{raw,processed}/
    line_size/{raw,processed}/
    associativity/{raw,processed}/
    inclusion_policy/{raw,processed}/
    latency/{raw,processed}/
    pmu/{raw,processed}/
  artemisia/
    README.md
    main_code/
    build/
    capacity/{raw,processed}/
    ...
  hazel_haswell/
    README.md
    slurm/
    main_code/
    build/
    ...
  hazel_genoa/
    ...

```

Use the same organization under `data_processed/` when practical. Every figure must be traceable to its input files: place a small **README**, manifest, or plotting command next to the processed data/plot that identifies the raw-data file(s), source file(s), exact compile command, compiler/version and flags, exact run command and inputs, affinity/Slurm binding, processing script, plotting script, relevant Git commit, machine, experiment parameters, exact sample count, units, random seed (when applicable), and any run that was excluded with the reason for exclusion. Prefer machine-readable formats (CSV/TSV, NumPy/binary plus a documented reader, or losslessly compressed equivalents). **Do not submit only screenshots, selected averages, or hand-copied summary values.**

The GitHub repository is required for code/history; you do not need to commit multi-gigabyte raw datasets to Git if doing so is impractical. The Moodle submission (or an instructor-approved alternate data handoff if Moodle limits are exceeded) is the authoritative raw-data submission.

13 Submission Checklist

- **One Moodle ZIP** containing the final report, final slides, main reverse-engineering code for every architecture used, scripts, and the required raw/processed data.
- **Complete main reverse-engineering source-code listings in the body of the PDF report**, including the architecture-specific x86-64 and AArch64 paths actually used. The printed listings must match the submitted ZIP/GitHub version.
- Phase-I timing-only source code and frozen raw results, including exact build/run commands, compiler/version/flags, timer method, affinity/binding, and the inspected disassembly excerpt for critical loops.
- Phase-II PMU/counter scripts and raw results.
- Eight-event cross-machine dataset with exact event names/semantics.
- Software-only hit-rate/residency estimator.
- Frozen lab-only Hazel prediction table, prediction plots, and the Git commit hash/timestamp recorded before Hazel testing.
- Slurm job scripts and held-out Hazel results for at least five CPU generations.
- Reverse-Engineering Scorecard for Competition 1, with the candidate/accepted machine-property results traceable to evidence.
- Competition 2 hit-rate validation table, final software-only estimator, frozen parameters, and Git commit hash.
- Hazel prediction summary table with frozen predicted values, Hazel measured values, a qualitative accuracy assessment, and a short explanation of successes/failures.
- Machine research table and credible references.
- Chronological master table sorted by microarchitecture/processor year.

- Complete Moore-style cross-generation plot set: L1/L2/LLC capacities, associativities, hit latencies, miss penalties, line size, inclusion/exclusion behavior, software cache metric, and selected PMU metrics.
- Cross-architecture Intel/AMD/Arm comparison using consistent markers/legend and uncertainty where applicable.
- At least one fitted trend/doubling-time or slope analysis where supported, plus one quantitative future prediction.
- At least **two named team cache laws** using both teammates' last names, with quantitative rules, Moore-style evidence plots, frozen Hazel predictions, and post-Hazel support/weakening/falsification analysis.
- A methodology flow chart/diagram set explaining the logic behind each reverse-engineering method.
- GitHub repository URL on the **first page of the report** and in README.md, with visible commit history and repository access granted to **both the TA and instructor**.
- GitHub contribution-plot screenshot in the report appendix showing **both students' GitHub accounts**, together with each student's full name and corresponding GitHub username.
- If used, reservation-sheet entries identifying the reserved lab machine/core/package and time window.
- Overleaf project URL on the **first page of the report** and in README.md, with project access granted to **both the TA and instructor**.
- Report titled *Predicting Cache Evolution: The Growing Capacity and Latency Cost of CPU Caching*.
- Final slide deck (PDF export) summarizing the project.
- PDF report, **all collected raw data**, processed data, plots, and reproduction instructions.
- Machine/server-organized data folders with plot-to-raw-data traceability (manifest/README or equivalent).
- Completed AI/LLM assistance disclosure identifying the tool and type of help received, if any.
- Contribution table and contributor labels for figures/tables.

14 Moodle Submission Details

The assignment is due **September 10, 2026 at 11:59 PM Eastern Time** on Moodle. The Moodle timestamp is the submission timestamp. Submit **one ZIP archive for the team**. The archive must open as one self-contained homework package containing the final report, final slides, the final main reverse-engineering code for all architectures used, and the main/raw/processed data required to reproduce the reported results. Verify the ZIP before the deadline; do not rely on GitHub or Overleaf as a substitute for files that are required in Moodle. **Large-data contingency:** because one-million-sample raw datasets can be large, use lossless compression. If the complete archive exceeds Moodle's upload limit, contact the TA **at least 24 hours before the deadline** for an alternate data-handoff method. Do not omit raw data or reduce the required sample count merely to fit an upload limit.

Recommended archive structure:

```
HW1_TeamName/
  README.md
  report/HW1_report.pdf
  slides/HW1_slides.pdf
  main_code/
    x86_64/                # Intel/AMD reverse-engineering source
    aarch64/              # Arm reverse-engineering source
    common/               # shared code, headers, common benchmark logic
    pmu/                  # Phase-II validation code/scripts, if separate
    software_hit_rate/    # final timing-only estimator
  scripts/                # processing/plotting/reproduction scripts
  data_raw/
    sunbird/{capacity,line_size,associativity,inclusion_policy,latency,pmu,...}/
    thunderbird/{capacity,line_size,associativity,inclusion_policy,latency,pmu,...}/
    skylark/{capacity,line_size,associativity,inclusion_policy,latency,pmu,...}/
    artemisia/{capacity,line_size,associativity,inclusion_policy,latency,pmu,...}/
    charnwood/{capacity,line_size,associativity,inclusion_policy,latency,pmu,...}/
    crux/{capacity,line_size,associativity,inclusion_policy,latency,pmu,...}/
    okay/{capacity,line_size,associativity,inclusion_policy,latency,pmu,...}/
    upgrade/{capacity,line_size,associativity,inclusion_policy,latency,pmu,...}/
    hazel_{<generation>}/{capacity,line_size,associativity,inclusion_policy,latency,...}/
  data_processed/
    <same machine-oriented organization>/
  plots/
  hpc_slurm/
```

The `README.md` must contain the GitHub URL, the Overleaf project link, confirmation that both were shared with the TA and instructor, build/run commands, machine-to-result mapping, and a short description of every script. The `main_code/` directory must contain the exact final main programs used to produce the submitted reverse-engineering results; those files must agree with the code listings printed in the body of the report. The archive must include **all collected experimental data**; at minimum, the main raw and processed datasets used by every reported figure/table must be immediately identifiable. Do not discard runs simply because they were noisy or were not selected for a final plot. If a run is invalid, keep it clearly labeled and explain why it was excluded from analysis.

15 Class Competitions and Leaderboards

Competition 1: Reverse-Engineering Championship — Who Can Reverse Engineer the Most?

The first competition rewards the team that produces the **largest number of valid, experimentally supported cache reverse-engineering results** across the available lab and Hazel machines.

A result counts as one **validated machine–property result** only when it is tied to a specific machine/CPU generation and is supported by the required evidence. Examples of countable cache properties include L1/L2/LLC capacity, cache-line size, associativity/effective associativity, number of sets, L1/L2/LLC hit latency, next-level or memory latency/miss penalty, and cache-sharing scope, and experimentally supported inclusion/exclusion behavior. Additional cache-related properties may count if approved by the instructor/TA.

A claimed number alone does not count. To be accepted for the leaderboard, each machine–property result must include:

- the microbenchmark/code and exact experiment parameters;
- the required raw data and repeated measurements;
- a plot/table showing the experimental signature used for the inference;
- a clear inference rule explaining why the data reveal that property; and
- verification with PMU/system/published information after the blind timing-only phase, when such verification is available.

The primary ranking is the **number of accepted validated machine–property results**. Repeating the same claim, splitting one result artificially into multiple rows, or reporting unsupported/specification-derived values does not increase the count. If teams tie, the instructor/TA will use, in order, broader machine/generation coverage, stronger cross-architecture coverage, reproducibility/evidence quality, and clarity of the report/presentation as tie-breakers.

Teams should include a one-page **Reverse-Engineering Scorecard** in the report appendix with one row per candidate result: machine, CPU generation, property, inferred value, evidence figure/table, validation source/result, and whether the team believes it should count. The instructor/TA determines the final accepted count.

Competition 2: Software-Only Cache Hit-Rate Championship

The second competition rewards the best **timing-only microbenchmark for approximating cache hit rate without reading hardware performance counters at runtime**. The submitted estimator must satisfy the requirements of the Software-Only Replacement section above.

For the official competition evaluation, the instructor/TA will run or review the submitted estimator on a standardized set of machine/workload combinations for which a trustworthy PMU-derived reference hit rate can be obtained. The official validation combinations may include cases that were not used by the team for calibration so that the competition measures generalization rather than memorization of one machine.

For each official validation case, define the absolute hit-rate deviation in **percentage points** as

$$E_i = \left| \hat{H}_i - H_i^{\text{PMU}} \right|,$$

where both rates are expressed on the same 0–100% scale. The primary competition score is the **mean absolute percentage-point deviation** across the official validation cases. The team with the **smallest** mean deviation wins. If two teams are effectively tied, the instructor/TA will use worst-case deviation, cross-machine robustness, runtime overhead, reproducibility, and quality of the methodology as tie-breakers.

Runtime restriction: the competition estimator itself may use timers and ordinary unprivileged instructions, but it may not read PMU counters, privileged registers, cache-topology answers, or external specification tables while producing its estimate. PMU data are used only as the reference for validation. Teams may use PMU data during the allowed Phase-II development/verification stage, but the final submitted estimator and parameters must be frozen in GitHub before official competition scoring.

Required Leaderboard Evidence

Every team must submit:

- a Reverse-Engineering Scorecard for Competition 1;
- the final software-only hit-rate estimator source code and frozen parameters for Competition 2;
- a validation table showing software-estimated versus PMU-derived hit rates on the team’s own available verification machines; and

- the Git commit hash identifying the competition-ready version of the code/results.

The two leaderboards are independent: a team may place in one, both, or neither. Ambitious teams are encouraged to compete seriously in both.

The Hazel prediction/reveal exercise remains mandatory for the scientific analysis. Every team must still include one summary table containing at least: target Hazel generation, frozen predicted value, later Hazel measured value, a concise assessment of prediction accuracy, and a short explanation. Also include at least one visual “prediction vs. reveal” plot with lab observations as solid points, the frozen model as a dashed line, and Hazel measurements as star/diamond markers. Do not retroactively alter the dashed prediction after Hazel is revealed.

16 Mandatory Team Contribution Appendix Template

Copy the following tables to the end of your report and complete them. This appendix is mandatory for every two-person team. Tables 6–14 make the work traceable from people to results, code, runs, writing, analysis, and AI assistance. It is intended to make the contribution to every major result traceable. Do not use vague statements such as “both contributed equally” without identifying what each person actually did.

A. Team Members and GitHub Accounts

Member	Name	Unity ID	GitHub username
Member 1			
Member 2			

Table 6: Team-member identification and GitHub accounts for the mandatory contribution appendix.

B. Result / Data-Point Attribution

For **every result shown in the paper**—including each final table entry, plotted machine/generation point, prediction point, PMU comparison, and major derived quantity—identify who produced the underlying measurement and who checked/processed it. Mechanically repeated points from one identical sweep may be grouped as one row (for example, “L1 capacity sweep on Sunbird, all working-set sizes”), but no reported result should be unattributed.

Table 7: Result/data-point attribution for every reported result.

Result / Data ID	Result, figure, table, machine, or data series	Member 1 contribution	Member 2 contribution	Raw-data / script / commit reference
R1				
R2				
R3				
R4				
R5				
R6				
R7				
R8				
R9				
R10				
R11				
R12				

Add rows as needed. Your report may require substantially more than 12 rows.

C. Team Meeting Log

Record project meetings or substantial synchronous work sessions, if any. Include who attended and a concise summary of decisions, work completed, unresolved issues, and next steps.

Table 8: Team meeting and synchronous-work log.

Date / Time	Purpose	Member 1 attended?	Member 2 attended?	Summary / decisions / next steps

Add rows as needed. If the team had no formal meetings, state that explicitly and describe how coordination occurred.

D. Code Contribution

List the code components, scripts, benchmark modules, build files, and infrastructure each member created, modified, reviewed, or debugged. Include Git commit hashes or pull-request references where useful.

Table 9: Code, script, build, and infrastructure contributions.

Code component / file / module	Member 1 contribution	Member 2 contribution	Git reference

Required GitHub contribution-plot screenshot

Immediately after the code-contribution table, insert a screenshot of the GitHub repository’s contributor/contribution plot (for example, **Insights** → **Contributors**) or an equivalent GitHub view that visibly shows both students’ accounts and contribution histories for this homework repository. The figure caption must map each GitHub username to the student’s **full name**. If a student’s commits do not appear because commits were made with an unlinked email/account, correct the repository/account linkage before submission when possible and explain any remaining discrepancy. This screenshot is supporting evidence only; the detailed tables and commit references remain mandatory.

Insert GitHub contributor/contribution plot screenshot here
Show both student accounts; caption with full name ↔ GitHub username mapping.

E. Experimental-Running Contribution

Document who actually ran each major experiment. Include the machine/generation and enough information to connect the work to the raw-data files. If both members ran independent repetitions for validation, state that explicitly.

Table 10: Experimental-running contribution by machine/generation.

Experiment / machine / generation	Run date(s)	Member 1 contribution	Member 2 contribution

F. Plotting and Writing Contribution

List each major figure/table and each major paper section. State who generated/checked the plots, who drafted the text, and who revised it.

Table 11: Plotting, table, and report-writing contributions.

Figure / table / report section	Member 1 contribution	Member 2 contribution

G. Analysis Contribution

Document the reasoning work separately from running experiments and making plots. Examples include identifying cache boundaries, deciding how associativity was inferred, interpreting distributions, validating PMU evidence, fitting chronological trends, explaining cross-architecture differences, and constructing/freeze-checking the Hazel prediction.

Table 12: Analysis and interpretation contributions.

Analysis question / conclusion	Member 1 contribution	Member 2 contribution

H. AI / LLM Assistance Disclosure

Complete this table if any AI/LLM tool was used. Add rows as needed. If no AI was used, write “None.” The purpose is to disclose **what kind of assistance was received**, not merely the name of the tool.

Table 13: AI/LLM assistance disclosure.

Tool	Task / artifact	Type of help received	What the team verified and can explain

Tool	Task / artifact	Type of help received	What the team verified and can explain

AI accountability statement: Both students are responsible for understanding every submitted artifact, including AI-assisted code and analysis. During grading or discussion, either team member may be asked to explain the code path, experimental logic, data-collection process, plotting pipeline, or reasoning behind a reported result.

I. Final Contribution Confirmation

Both members must review the completed appendix before submission and complete Table 14.

Member	Name	Confirmation / Date
Member 1		
Member 2		

Table 14: Final contribution-appendix confirmation by both team members.

Foundational Trend Reference

- G. E. Moore, “Cramming More Components onto Integrated Circuits, *Electronics*, April 19, 1965; reprinted in *Proceedings of the IEEE*, vol. 86, no. 1, 1998. The original Figure 3 reproduced earlier in this homework is used only to illustrate the observed-solid / predicted-dashed presentation style.

Course and Infrastructure References

- Course slide: *Compute Resources Available to Students*, listing Sunbird, Thunderbird, Skylark, Artemisia, Charnwood, Crux, Ookay, and Upgrade and their CPU models.
- NC State High Performance Computing, *HPC Resources*: <https://hpc.ncsu.edu/Resources/Resources.php>
- NC State High Performance Computing, *Slurm Partitions and Resources*: <https://hpc.ncsu.edu/RunningJobs/Resources.php>
- NC State High Performance Computing, *Slurm Deployment Status*: <https://hpc.ncsu.edu/RunningJobs/Status.php>
- NC State High Performance Computing, *Request Access / Instructional Use*: <https://hpc.ncsu.edu/Accounts/GetAccess.php>
- NC State High Performance Computing, *Logging in to Hazel*: <https://hpc.ncsu.edu/GettingStarted/Connecting.php>
- Homework I optional lab-machine reservation sheet: <https://docs.google.com/spreadsheets/d/1LJsgbHEbgabqDhfg50q89RXtVYgDLgztwGntYXHCKFw/edit?usp=sharing>
- NC State High Performance Computing, *Understanding the Cluster*: <https://hpc.ncsu.edu/GettingStarted/Concepts.php>

Measurement Methodology References

Use these references to understand the timing and tooling examples above. They are methodology references; during Phase I, do not use any source that reveals the cache specifications you are supposed to infer.

- Intel, *Intel 64 and IA-32 Architectures Software Developer’s Manual*, especially the RDTSC, RDTSCP, LFENCE, and time-stamp-counter descriptions. The RDTSC instruction reference explicitly notes that it is not serializing and describes fence sequences for ordering. <https://cdrdv2-public.intel.com/835781/325462-sdm-vol-1-2abcd-3abcd-4.pdf>
- Arm, *AArch64 Programmer’s Guide: Generic Timer*, for the architectural counter/timer model, CNTVCT_ELO, and CNTFRQ_ELO. <https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/Learn%20the%20Architecture/Generic%20Timer.pdf>
- Linux manual page, `perf_event_open(2)`, for opening, grouping, enabling, disabling, and reading performance-monitoring events from C. https://man7.org/linux/man-pages/man2/perf_event_open.2.html

- Linux manual page, `taskset(1)`, for CPU affinity.
<https://man7.org/linux/man-pages/man1/taskset.1.html>
- GNU GCC documentation, *Options That Control Optimization*, for the meaning of `-O0` and optimization levels.
<https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html>
- GNU Binutils documentation, `objdump`, especially `-d/-disassemble` and `x86 -M intel`.
<https://sourceware.org/binutils/docs/binutils/objdump.html>
- A. Fog, *The Microarchitecture of Intel, AMD, and VIA CPUs*. Use this only after the relevant timing-only inference is frozen. It is useful for cache geometry/latency validation and for documented memory-disambiguation, false-dependence, and store-forwarding effects across x86 microarchitectures.
<https://www.agner.org/optimize/microarchitecture.pdf>

Optional Background and Reverse-Engineering Reading

These readings are provided as optional references when you need ideas for experimental design, timing methodology, cache behavior, PMU interpretation, or broader microarchitecture reverse engineering. You are not expected to reproduce the attacks in these papers. For Homework I, start with the cache/memory-hierarchy group below; the later groups are useful for learning how other microarchitectural structures have been reverse engineered.

A. Cache, Prefetching, TLB, and Memory-Hierarchy Methods — Most Relevant

- [1] X. Yu et al., “IMP: Indirect Memory Prefetcher,” *MICRO*, 2015.
<http://dx.doi.org/10.1145/2830772.2830807>
<https://pages.cs.wisc.edu/~yxy/pubs/imp.pdf>
- [2] J. Vicarte et al., “Augury: Using Data Memory-Dependent Prefetchers to Leak Arbitrary Data.”
<https://www.prefetchers.info/augury.pdf>
- [3] B. Chen et al., “GoFetch: Leaking Secrets via Microarchitectural Prefetching on Apple Silicon,” *USENIX Security*, 2024.
<https://www.usenix.org/system/files/usenixsecurity24-chen-boru.pdf>
- [4] Y. Yarom and K. Falkner, “FLUSH+RELOAD,” *USENIX Security*, 2014.
<https://www.usenix.org/system/files/conference/usenixsecurity14/sec14-paper-yarom.pdf>
- [5] E. Tromer, D. A. Osvik, and A. Shamir, “Efficient Cache Attacks on AES,” *Journal of Cryptology*, 2009.
<https://cs-people.bu.edu/tromer/papers/cache-joc-official.pdf>
- [6] B. Gras et al., “Defeating Cache Side-channel Protections with TLB Attacks (TLBleed),” *USENIX Security*, 2018.
<https://www.usenix.org/system/files/conference/usenixsecurity18/sec18-gras.pdf>
- [7] M. Yan et al., “Attack Directories, Not Caches: Side-Channel Attacks in a Non-Inclusive World,” *IEEE S&P*, 2019.
<https://people.csail.mit.edu/mengjia/data/2019.Oakland.DirectoryAttack.pdf>
- [8] D. Gruss et al., “Cache Template Attacks,” *USENIX Security*, 2015.
<https://gruss.cc/files/cta.pdf>
- [9] M. Kayaalp et al., “A High-Resolution Side-Channel Attack on Last-Level Cache,” *DAC*, 2016.
<https://www.cs.ucr.edu/~nael/pubs/dac16.pdf>
- [10] S. van Schaik et al., “Reverse Engineering Hardware Page Table Caches Using Performance Counters,” 2017.
https://download.vusec.net/papers/revanc_ir-cs-77.pdf
- [11] D. Weber, D. Gruss, and M. Schwarz, “Automated Discovery of Microarchitectural Side Channels,” *USENIX Security*, 2021.
<https://www.usenix.org/system/files/sec21-weber.pdf>

B. General Microarchitecture Measurement and Reverse Engineering

- [12] H. Wong, “Measuring Reorder Buffer Capacity,” Stuffed Cow Blog, 2013.
<https://blog.stuffedcow.net/2013/05/measuring-rob-capacity/>
- [13] T. Downs, “robsize: ROB size testing utility,” GitHub.
<https://github.com/travisdowns/robsize>
- [14] T. Downs, “A Note on Mask Registers,” Performance Matters blog, 2019.
<https://travisdowns.github.io/blog/2019/12/05/kreg-facts.html>
- [15] A. Fog, “The Microarchitecture of Intel, AMD, and VIA CPUs,” 2026. This manual is a key *post-Phase-I* validation source for x86 cache geometry and latency; do not use it to reveal blind/Hazel answers early.
<https://www.agner.org/optimize/microarchitecture.pdf>
- [16] A. Fog, “Instruction Tables: Latencies, Throughputs, μ op breakdown,” 2025.
https://www.agner.org/optimize/instruction_tables.pdf

- [17] A. Fog, “Software Optimization Resources.”
<https://www.agner.org/optimize/>
- [18] A. Fogh, “Covert Shotgun” (blog), 2016.
<https://cyber.wtf/2016/09/27/covert-shotgun/>

C. Branch Prediction, Speculation, and Related Reverse Engineering

- [19] D. A. Jiménez and C. Lin, “Dynamic Branch Prediction with Perceptrons,” *HPCA*, 2001.
<https://doi.org/10.1109/HPCA.2001.903263>
<https://www.cs.utexas.edu/~lin/papers/hpca01.pdf>
- [20] A. Sezne, “The L-TAGE Branch Predictor,” *JILP*, 2007.
<https://jilp.org/vol9/v9paper6.pdf>
- [21] H. Yavarzadeh et al., “Half&Half: Demystifying Intel’s Directional Branch Predictors for Fast, Secure Partitioned Execution,” *ASPLOS*, 2023.
<https://cseweb.ucsd.edu/~dstefan/pubs/yavarzadeh:2023:half.pdf>
- [22] L. Li et al., “Indirector: High-Precision Branch Target Injection Attacks and Reverse Engineering of Indirect Branch Prediction,” *USENIX Security*, 2024.
<https://www.usenix.org/system/files/usenixsecurity24-li-luyi.pdf>
- [23] N. Mahling, “Reverse Engineering of Intel’s Branch Prediction,” Bachelor’s Thesis, Univ. Lübeck (ITS), 2023.
https://www.its.uni-luebeck.de/fileadmin/files/theses/BA_NickMahling_ReverseEngineeringIntelsBranchPrediction.pdf
- [24] P. Kocher et al., “Spectre Attacks: Exploiting Speculative Execution.”
<https://spectreattack.com/spectre.pdf>
- [25] D. Evtushkin et al., “BranchScope: A New Side-Channel Attack on Directional Branch Predictor,” *ASPLOS*, 2018.
<https://www.cs.ucr.edu/~nael/pubs/asplos18.pdf>
- [26] A. Bhattacharyya et al., “SMoTherSpectre: Exploiting Speculative Execution through Port Contention,” 2019.
<https://arxiv.org/abs/1903.01843>

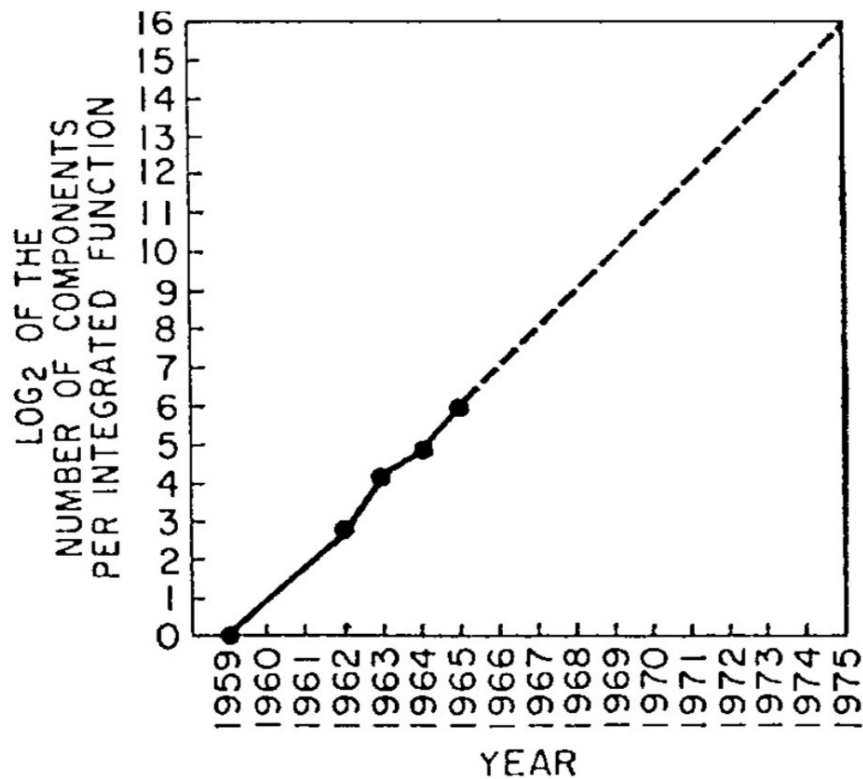


Figure 4: Moore-style plot reminder. Figure 3 from Gordon E. Moore, “Cramming More Components onto Integrated Circuits (1965; reprinted in *Proceedings of the IEEE*, 1998). The observed trend is shown with a **solid line/solid measured points**, followed by a **dashed extrapolation only for the future years** beyond the observed data. Your cache-evolution plots do not need to match the exact numbers or axis labels here, but the visual logic should be similar when you present measured observations and then a prediction.

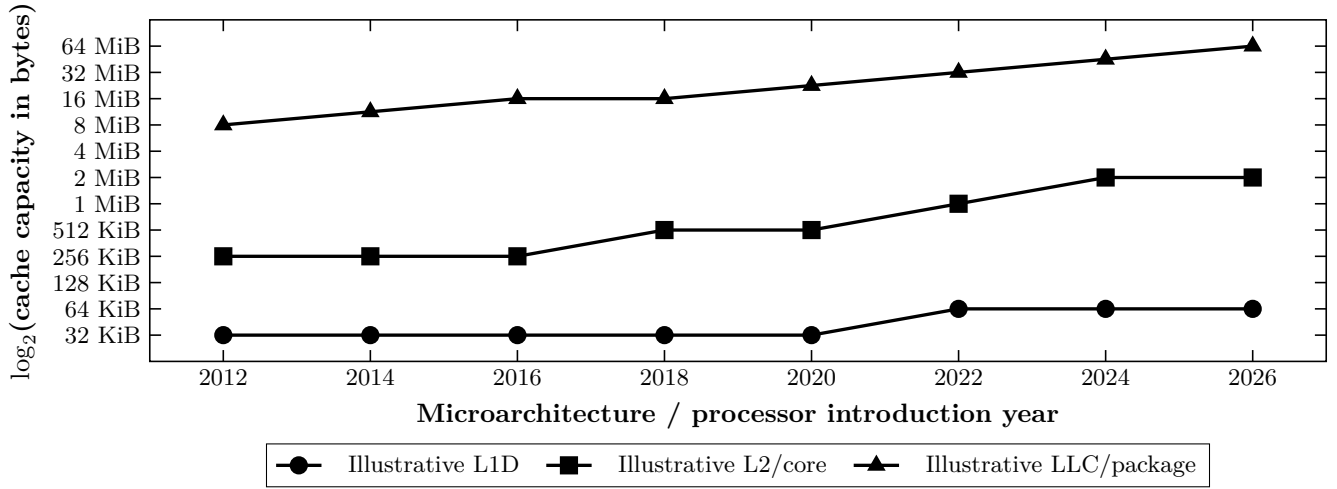


Figure 5: Illustrative Moore-style chronological plot. This is a schematic example only. Put microarchitecture introduction year on the x-axis and a cache quantity on the y-axis. For cache capacity, a $\log_2$ scale is especially useful because equal vertical steps correspond to doubling. Your final plots must contain the measured machines, identify architecture/vendor, show uncertainty where applicable, and must not copy these example values.

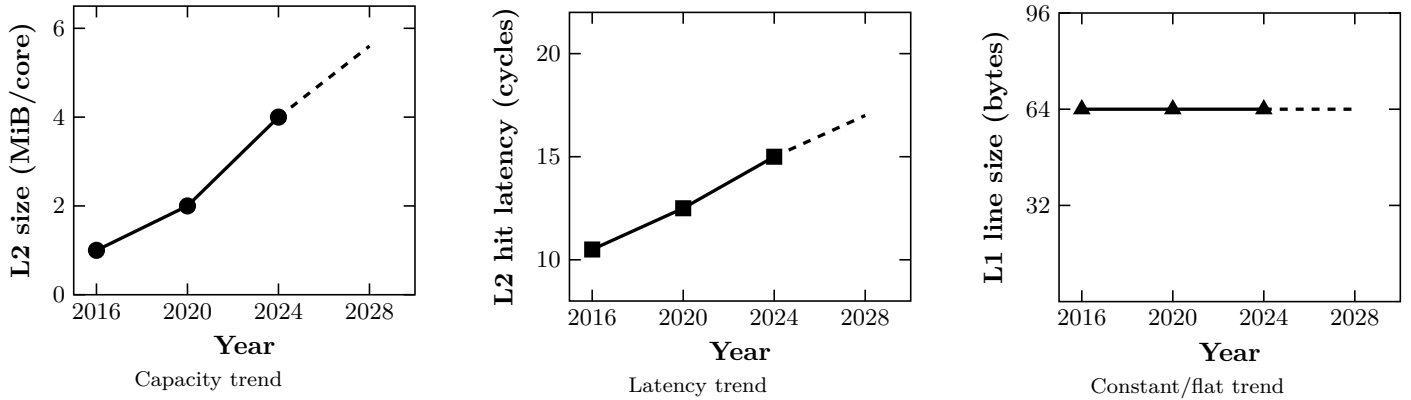


Figure 6: Illustrative predictive plot format. Each predictive series should show **at least three measured observation points connected by a solid line** ordered by year, followed by a **dashed trend/projection beginning at the last measured point** extending beyond the newest observation. The dashed line should represent the model or extrapolation you are using (linear, stepwise, doubling-like, or flat), and the caption/text must explain why that model is defensible. Do not invent a projection when you have fewer than three observations for the plotted series.

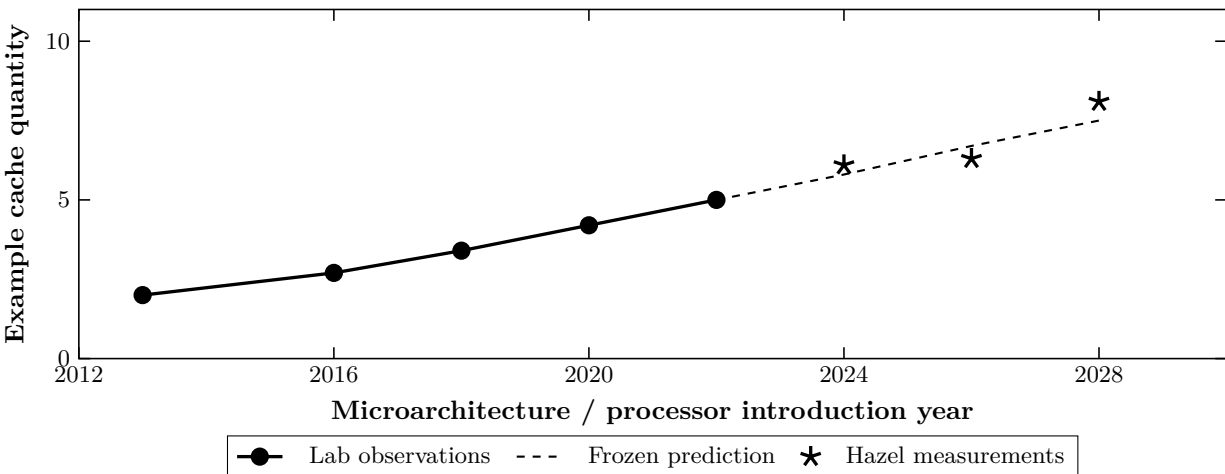


Figure 7: Illustrative held-out prediction plot. **Solid dots connected by a solid line** are observations from the instructor's ECE/lab machines and are the only cache measurements allowed when fitting the prediction. The **dashed line begins at the last lab observation and extends only into future/unmeasured years**; it is the prediction that must be frozen before Hazel testing. The **star markers** are the later Hazel reverse-engineering measurements used as held-out validation. Values shown here are schematic only.

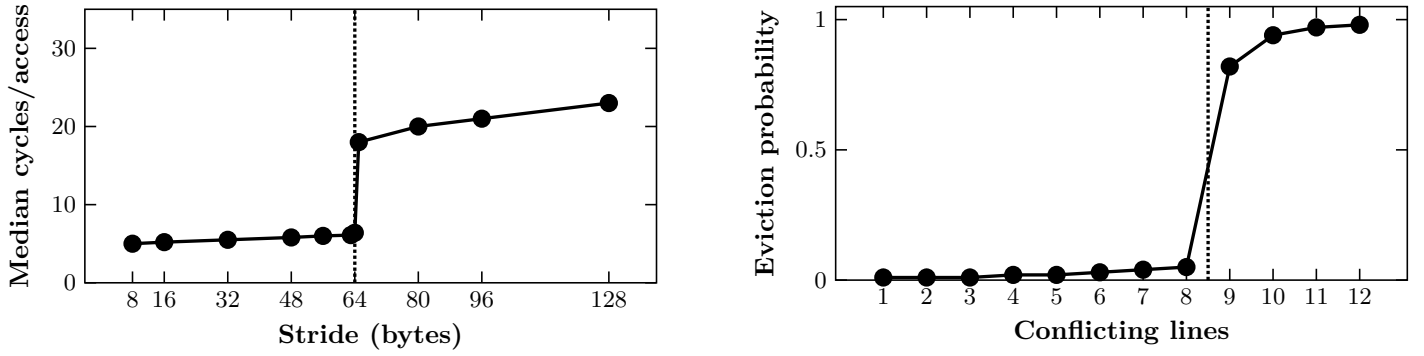


Figure 8: Illustrative sharp transitions for line-size (left) and associativity (right). In the associativity example, eviction begins when the conflict set exceeds eight resident lines, suggesting an 8-way structure.

17 Illustrative Result Shapes (Examples Only)

The following figures are **schematic examples, not expected numerical answers**. Your machine may have different sizes, latencies, and associativities. The purpose is to show the type of evidence that makes a reverse-engineering claim convincing. Your data should determine the answer; do not tune the data to resemble these examples.

In particular, when you make a chronological prediction plot, we want it to visually resemble the classic Moore-style format: **solid measured data points connected by a solid line first, then a dashed continuation only for the future projection**. Figure 4 is the historical reminder; Figure 5 shows a cache-oriented chronological layout; Figure 6 shows several allowable trend shapes; and Figure 7 shows the required lab-observation → frozen-prediction → Hazel-reveal logic. Figure 8 and Figure 9 illustrate the shape of sharp reverse-engineering evidence. All values in these examples are schematic.

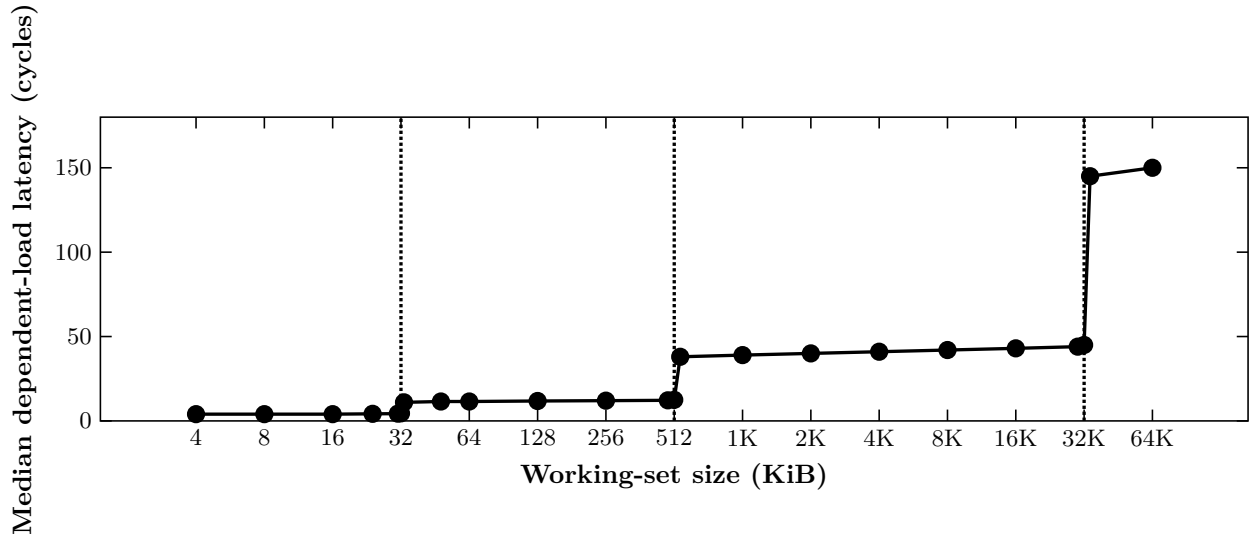


Figure 9: Illustrative capacity/hierarchy curve. Dense points near each boundary reveal sharp latency steps. The dotted vertical lines are example inferred boundaries only; dashed lines are reserved for future-year prediction/extrapolation in chronological figures.